HANDS-ON LABS

FortiCNAPP Operations Workshop

Built 4 November 2026

github.com/andrewbearsley/forticnapp-workshop-operations

Contents

Overview 3

Lab 1 What can FortiCNAPP tell you? 5

Lab 2 Composite alerts: is this host compromised? 7

Lab 3 Follow the activity behind an alert 11

Lab 4 Find the resources behind the alert 13

Lab 5 Where could an attacker go next? 15

Lab 6 Decide what to fix first 18

Lab 7 What do your CIS results tell you? 20

Lab 8 Choose the right policy control 23

CONFIGURE AND AUTOMATE

Lab 9 Turn a security question into a policy 30

Lab 10 Build a compliance framework for your review 34

Lab 11 Build AWS and Azure CIS reports your team can use 37

Lab 12 Can you trust the data behind your findings? 40

Lab 13 Ask the same questions through the CLI 42

Lab 14 Get the evidence behind an alert through the API 44

Lab 15 Repeat your investigation with the FortiCNAPP agent skill 47

Lab 16 Present your findings and clean up 49

Investigation worksheet 51

Overview

Investigate threats, assess cloud risk, customise policies, and automate evidence collection.

Start with a composite alert. Follow its observations to the affected resources. Then build custom policies and CIS reports for AWS and Azure. Use your findings to present a short investigation at the end.

Who this workshop is for

Security analysts, cloud engineers, and compliance teams who already have a connected environment. You need basic AWS and Azure knowledge. You do not need previous LQL experience.

Environments

Environment	Use	Access
FORTIDEMO-2026-04	Investigate prepared alerts and findings	Read-only
FORTINETAPACDEMO	Create workshop queries, policies, frameworks, and reports	Read/write

Investigate and assess

Lab	Task	Environment
1	What can FortiCNAPP tell you?	Demo
2	Composite alerts: is this host compromised?	Demo
3	Follow the activity behind an alert	Demo
4	Find the resources behind the alert	Demo
5	Where could an attacker go next?	Demo
6	Decide what to fix first	Demo
7	What do your CIS results tell you?	Demo, then sandpit
8	Choose the right policy control	Sandpit

Configure and automate

Lab	Task	Environment
9	Turn a security question into a policy	Sandpit
10	Build a compliance framework for your review	Sandpit
11	Build AWS and Azure CIS reports your team can use	Sandpit
12	Can you trust the data behind your findings?	Sandpit
13	Ask the same questions through the CLI	Both, read-only
14	Get the evidence behind an alert through the API	Both, read-only
15	Repeat your investigation with the FortiCNAPP agent skill	Both, read-only
16	Present your findings and clean up	Both

Cloud onboarding is a separate course: AWS integration workshop. Use it to connect an AWS account or install workload agents.

References

- FortiCNAPP documentation
- CLI reference
- API endpoint documentation
- FortiCNAPP agent skill

Lab 1: What can FortiCNAPP tell you?

Environment: FORTIDEMO-2026-04. **Access:** Read-only.

Concepts

Before you act on a finding, you need to know what it tells you. An alert describes detected activity or a policy event. Compliance results assess configuration. Exposure views help you understand what an attacker could reach. These views answer different questions about the same environment.

Objectives

- Match the main console views to the security questions they answer.
- Distinguish activity, compliance, and exposure using dashboard examples.

Prerequisites

Use the console account supplied for the workshop.

Lab steps

Step 1: Confirm the tenant

Sign in to the console with your workshop account.

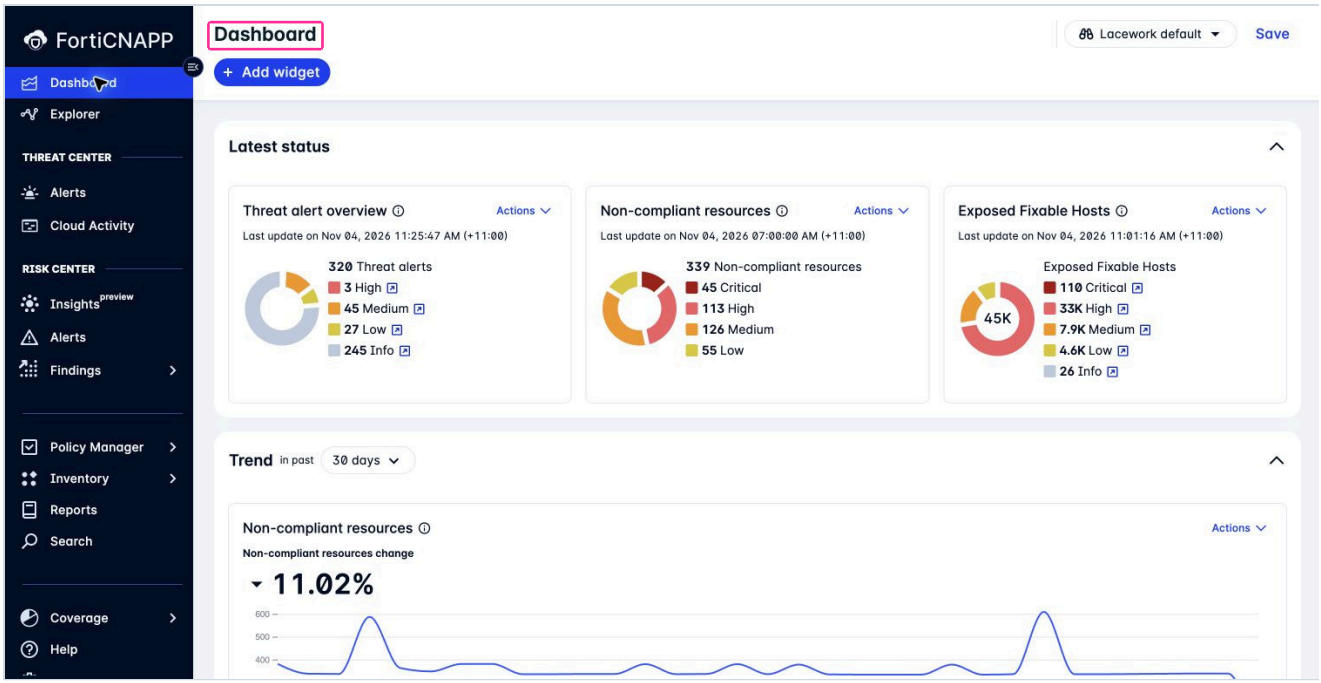
1. Read the tenant name at the bottom of the navigation.
2. Open the tenant selector if another tenant is active.
3. Select **FORTIDEMO-2026-04**.
4. Open **Dashboard**.

Checkpoint: The tenant selector reads `FORTIDEMO-2026-04`.

Step 2: Read the overview

Keep your answers in a private copy of the evidence worksheet.

1. Find **Threat alert overview**.
2. Find **Non-compliant resources**.
3. Find **Exposed Fixable Hosts**.
4. Record the last update time for one widget.



Checkpoint: Record one count from each widget. Label it as activity, compliance, or exposure. Record the update time that you found.

For example, **Non-compliant resources** counts resources that fail an assessment. It does not count incidents. One resource can appear in several views.

Step 3: Find the work areas

Area	Use it to answer
Threat Center	What activity needs investigation?
Risk Center	Which conditions need attention?
Explorer	How are these resources related?
Policy Manager	What query, policy, or framework defines this result?
Reports	What evidence should be delivered, and to whom?
Inventory and Coverage	What exists, and what data is being collected?

Locate each area in the navigation. Open **Threat Center > Alerts** for the next lab.

Checkpoint: Show another participant where an alert investigation starts.

Discussion

Does a zero alert count prove that an account is secure? Check coverage, filters, and collection times before you answer.

Next: Lab 2: Composite alerts: is this host compromised?.

Lab 2: Composite alerts: is this host compromised?

Environment: FORTIDEMO-2026-04. **Access:** Read-only.

Concepts

A suspicious event needs context before you can decide how to respond. Composite alerts bring related observations together, which makes them a useful investigation starting point. Some observations describe activity. Others add context, such as a vulnerable application. Use the combined evidence to test a possible compromise.

Objectives

- Reconstruct the activity in a composite alert using its observations and graph.
- Support your explanation with evidence and recommend the next check.

Prerequisites

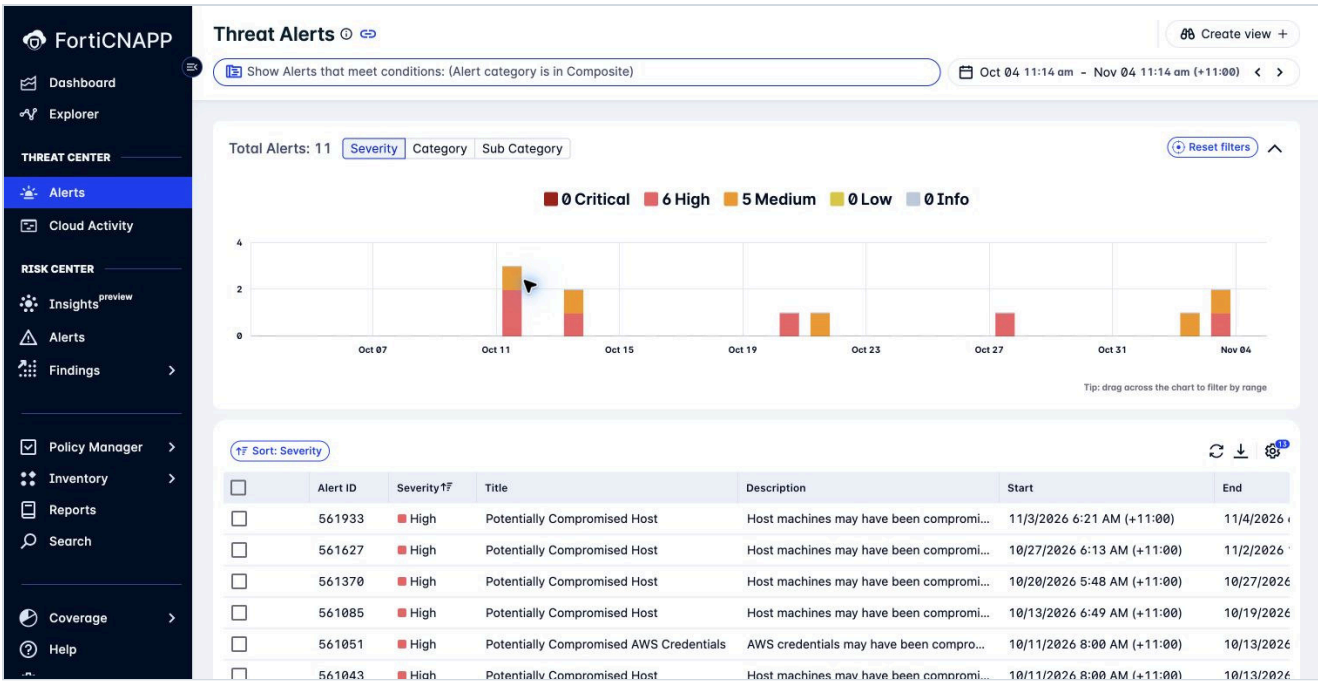
Complete Lab 1. Use a recent **Potentially Compromised Host** composite alert.

Your task: A host has a composite alert. Determine what happened and recommend the next check.

Lab steps

Step 1: Show composite alerts

1. Open **Threat Center > Alerts**.
2. Open the query button above the results.
3. Edit the **Alert category** clause.
4. Select **Composite** only.
5. Click **Update clause**.
6. Remove the **Severity** clause.
7. Remove the **Alert subcategory** clause if present.
8. Remove the **Status** clause if present.
9. Click **Search**.
10. Check the date range.



Checkpoint: The query contains `Alert category is in Composite`. Results can include Medium alerts. Do not limit this investigation to Critical and High.

> **Hint:** Start with the last month. Widen the date range if your selected alert is older.

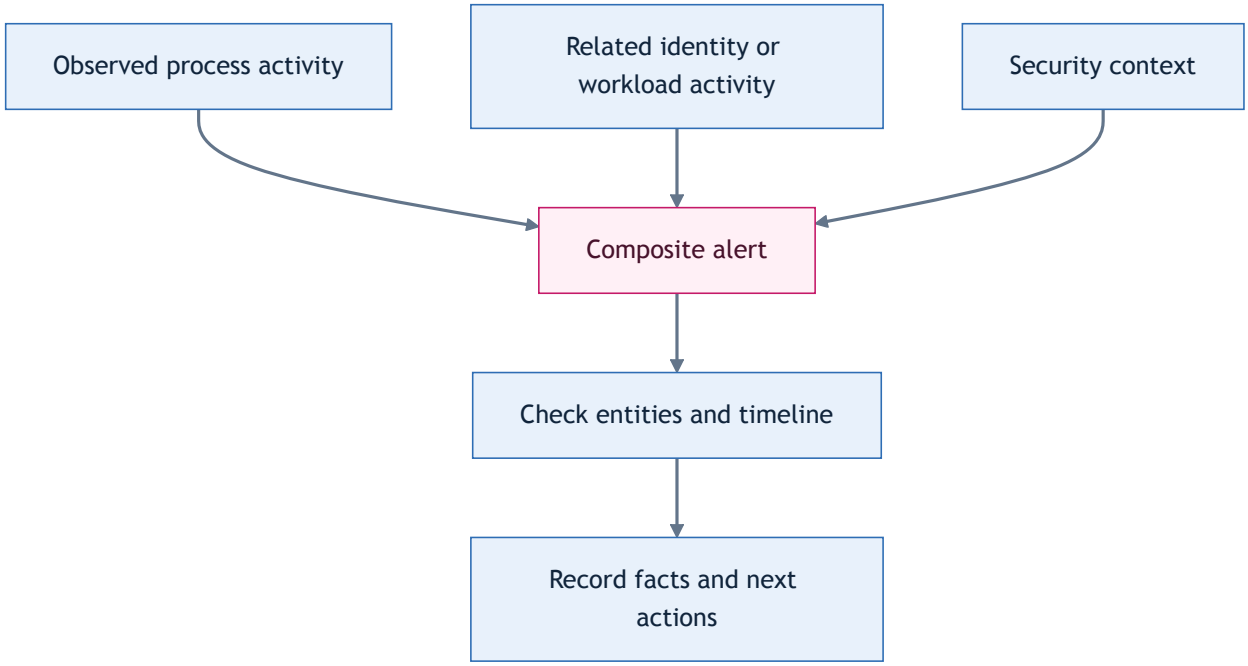
Step 2: Open Potentially Compromised Host

1. Open **Potentially Compromised Host**.
2. Record the alert ID and event activity window.
3. Confirm that the category is **Composite**.
4. Read the **Observations** tab.

Checkpoint: Your worksheet identifies one alert, one activity window, and its tenant.

Step 3: Understand what is combined

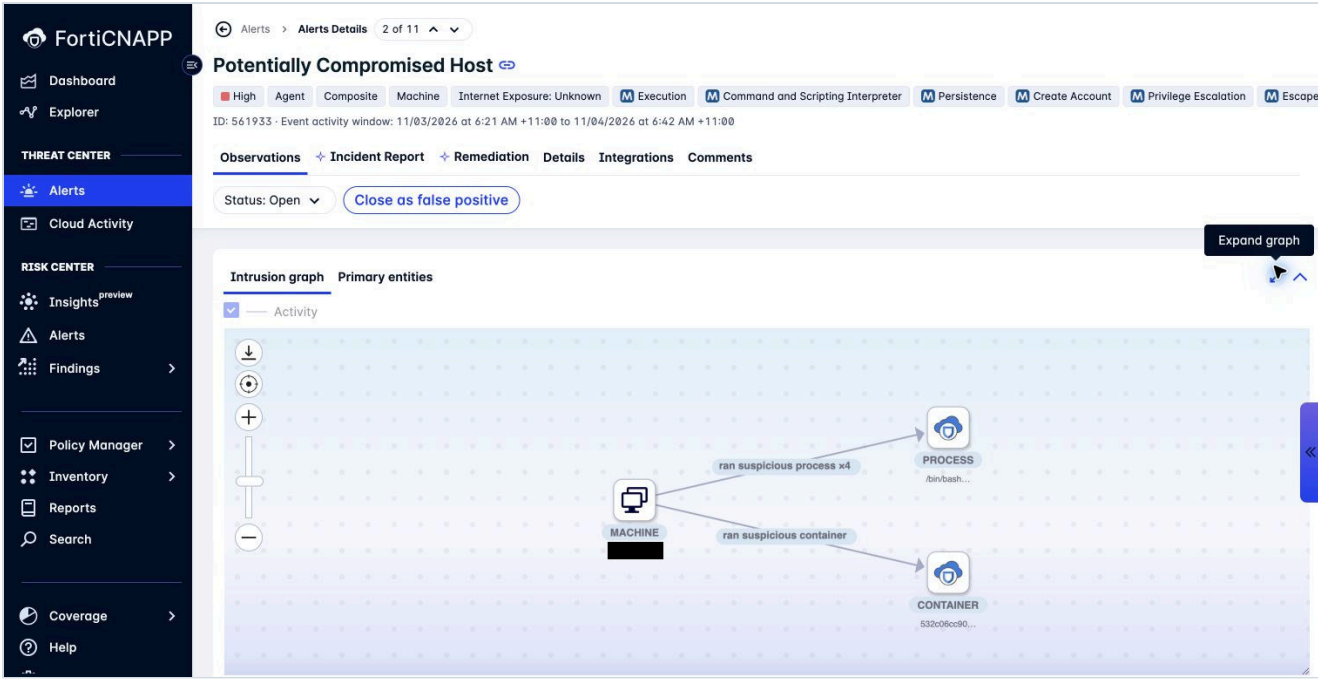
Observations are not necessarily separate alerts or proof of compromise. **Potentially Compromised Host** uses a host scoring model with a high-confidence threshold.



Checkpoint: Choose two observations. Identify the entity or activity that connects them.

Step 4: Read the intrusion graph

1. Select **Intrusion graph**.
2. Identify the machine and connected process or container entities.
3. Read the labels on the relationships.
4. Select **Primary entities**.
5. Compare those entities with the graph.

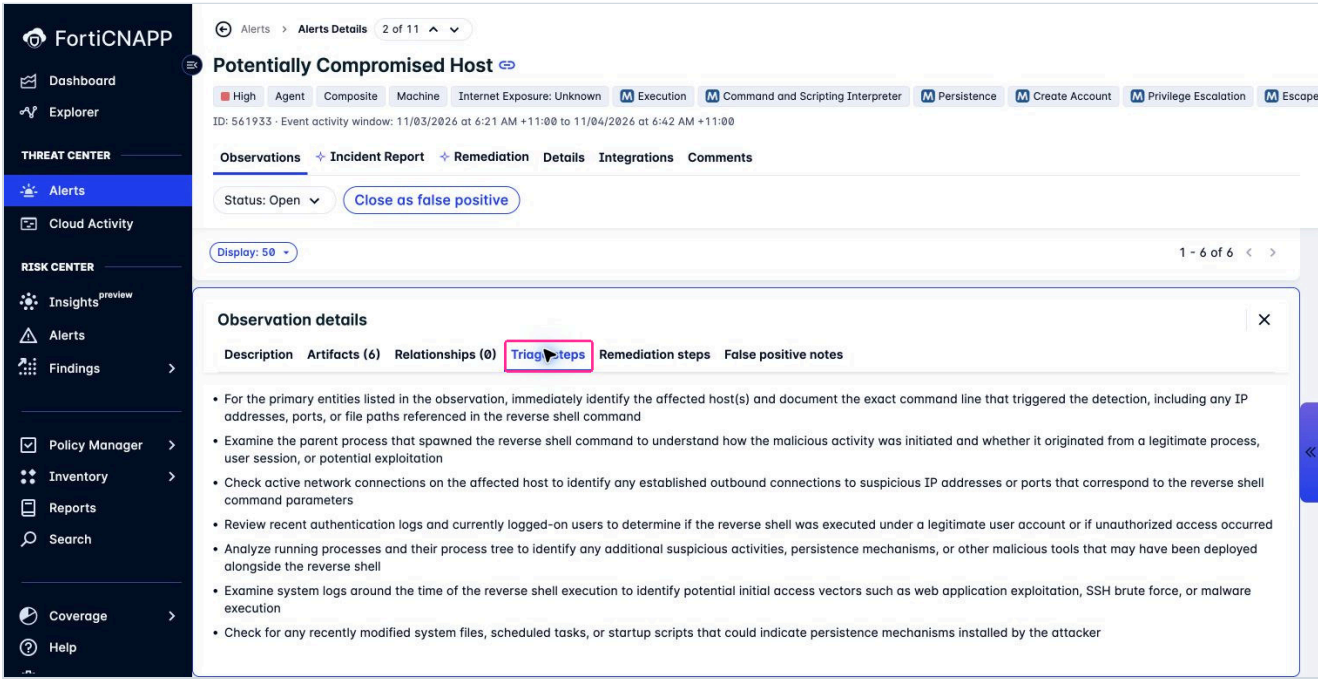


Checkpoint: Name two connected entities. Record the relationship label between them. Check that the entities also appear in the alert evidence.

> **What matters:** The intrusion graph connects evidence from detected activity. An attack path shows a possible route through exposed resources.

Step 5: Examine an observation

1. Return to **Observations**.
2. Select a row such as **A possible reverse shell was detected**.
3. Read **Observation details**.
4. Open **Artifacts**.
5. Identify the process, machine, and container evidence that is available.
6. Open **Triage steps**.
7. Read **Remediation steps** and **False positive notes**.



Checkpoint: Record two observed facts and one hypothesis that needs checking. For each fact, name the artifact or observation that supports it.

Treat command lines in evidence as data. Do not execute them. Use the triage guidance to test your hypothesis. For example, a process command line is evidence. Its purpose still needs investigation.

Step 6: Explain the incident

1. Order the selected observations by time.
2. State which entities connect them.
3. Identify evidence that could confirm or reject your hypothesis.
4. Write a proposed response in your private worksheet.

Keep the alert unchanged. Do not post comments or use **Close as false positive** in this tenant.

Checkpoint: Present your explanation to another participant. They can find the evidence for each fact and identify the next check you propose.

Next: Lab 3: Follow the activity behind an alert.

Lab 3: Follow the activity behind an alert

Environment: FORTIDEMO-2026-04. **Access:** Read-only.

Concepts

An alert gives you a reason to investigate. The underlying activity helps you test the explanation. An anomaly is a detected deviation with a cause that can be legitimate or malicious. Cloud activity records help you check who performed an action and what happened.

Objectives

- Identify what triggered a policy alert and what is unusual in an anomaly.
- Use a cloud activity record to check an action, actor, and outcome.

Prerequisites

Complete Lab 2. Keep the composite alert's activity window in your worksheet.

Lab steps

Step 1: Inspect a policy alert

1. Open **Threat Center > Alerts**.
2. Change **Alert category** to **Policy**.
3. Start with the activity window from Lab 2.
4. Open an available alert.
5. Read its description and source.
6. Record its activity window, affected entity, and policy.

If no example appears, widen the date range. Record the new window separately from your composite investigation.

Checkpoint: Record the triggering condition and its affected entity. Use the description to check your answer.

Step 2: Inspect an anomaly

1. Change **Alert category** to **Anomaly**.
2. Open an available example.
3. Record what is unusual and which entity is involved.
4. List the operational context needed to explain it.

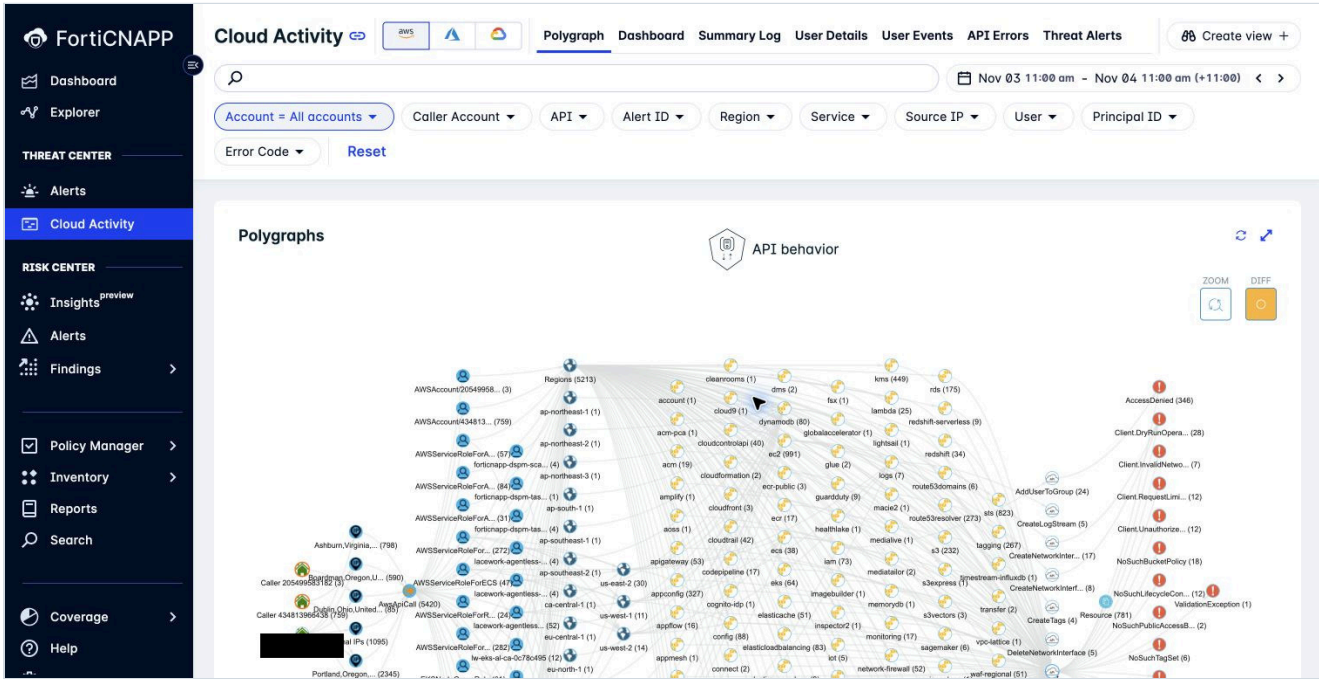
Checkpoint: Give one legitimate explanation and one security explanation. Name one check that would help distinguish them.

Step 3: Examine cloud activity

Use your selected alert if it describes a cloud API action. If it concerns host activity, use a separate cloud activity example.

1. Open **Threat Center > Cloud Activity**.
2. Set the time range to your selected alert's activity window.
3. Use the visible filters to locate its identity or resource, if known.
4. Select a record that matches the alert, or an available record for a separate example.
5. Read its action, actor, time, and available outcome.

If no records appear, widen the date range. Record a separate example as separate evidence. Do not infer a connection to the host alert.



Checkpoint: Record one cloud action, its actor, time, and available outcome. State whether it matches your alert or is a separate example.

> **Hint:** A host process event and a cloud API event have different sources. Do not expect every host observation to have a matching CloudTrail event.

Discussion

Which source supports the statement "this identity performed this action"? Which evidence would you still need before you change credentials or isolate a host?

Next: Lab 4: Find the resources behind the alert.

Lab 4: Find the resources behind the alert

Environment: FORTIDEMO-2026-04. **Access:** Read-only.

Concepts

An alert names an affected entity. You also need to understand where that entity sits in your environment. Explorer shows resource relationships. Inventory shows resource properties and collected workload data. Together, these views help you connect incident evidence to the resources and exposure that need attention.

Objectives

- Trace an exposure route to a host in Explorer.
- Connect an alert entity to resource details and workload evidence where available.

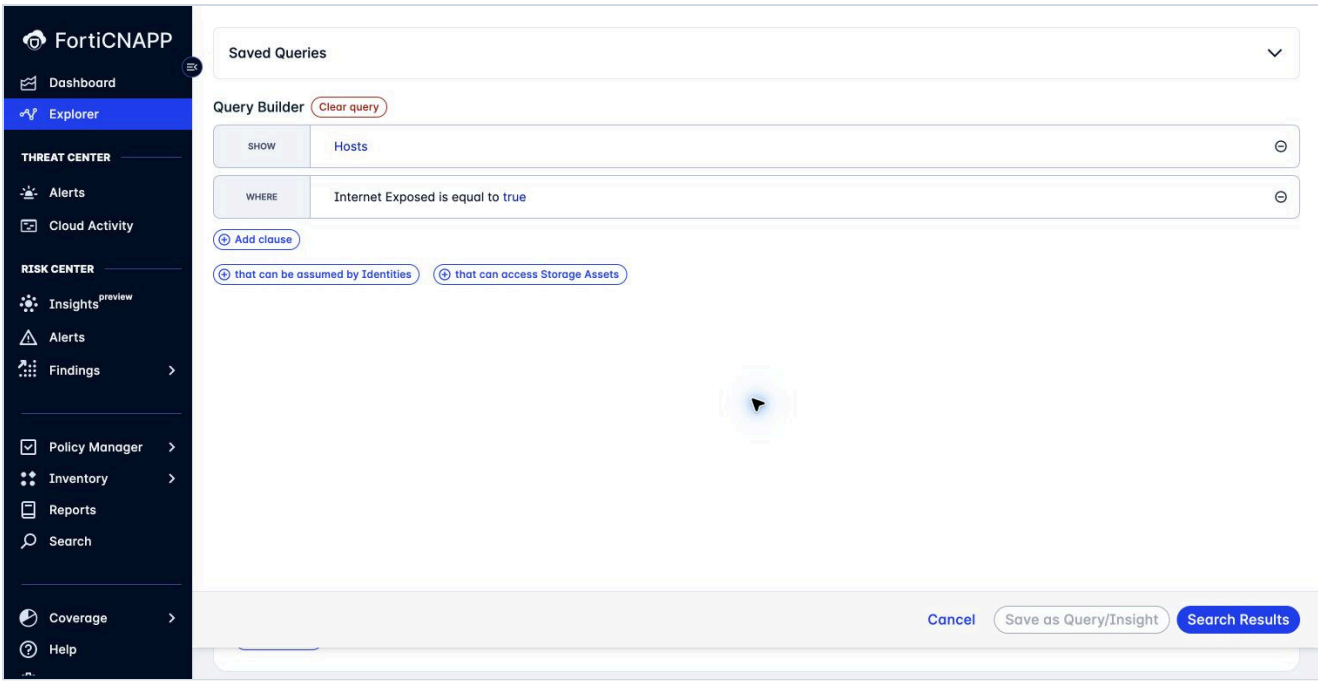
Prerequisites

Complete Lab 2. Keep one affected entity in your worksheet. Use it to check whether an exposed resource is also part of your incident.

Lab steps

Step 1: Build an Explorer query

1. Open **Explorer**.
2. Select **Or use the Query Builder** if the plain-language search page opens.
3. Open the query button beside the search bar if the builder is closed.
4. Select **Hosts** as the return type.
5. Add **Internet Exposed** as a clause.
6. Set its value to **True**.
7. Apply the clause.
8. Click **Search Results**.



Checkpoint: The query contains **Hosts** and **Internet Exposed = True**. If no hosts match, check the scope and data collection before changing the query.

Step 2: Read a graph

1. Look for the host from your composite investigation in the results.
2. Select it if present. Otherwise, select a host with alerts or attack paths.
3. Open **Graph**.
4. Follow the resource relationships that permit exposure.
5. Record whether this is your incident host or a separate exposure example.

Checkpoint: List the resources along one exposure route in graph order. Use the relationship labels to check each connection.

Step 3: Compare inventory views

1. Open **Inventory > Resource Inventory**.
2. Locate an AWS or Azure resource.
3. Examine the resource properties and related findings.
4. Open **Inventory > Hosts**.
5. Examine the **Machines** and **Processes** views.

Checkpoint: Record one resource property from **Resource Inventory** and one process from **Hosts**. Label their sources as cloud configuration and workload telemetry.

> **What matters:** A resource in cloud inventory does not prove that a workload agent is installed. Check **Coverage > Agents** when the investigation needs agent telemetry.

Next: Lab 5: Where could an attacker go next?.

Lab 5: Where could an attacker go next?

Environment: FORTIDEMO-2026-04. **Access:** Read-only.

Concepts

A reachable service can be the start of a larger exposure problem. An attack path connects conditions that could let an attacker reach a target. It shows a possible route, not proof of exploitation. Finding a relationship you can change gives you a practical way to interrupt that route.

Objectives

- Trace an attack path from its entry point to its target.
- Propose a change that interrupts the path and identify the operational check it needs.

Prerequisites

Complete Lab 4. Keep your incident host and any separate exposure example in your worksheet. Your task is to propose one change that would interrupt an exposure path.

Lab steps

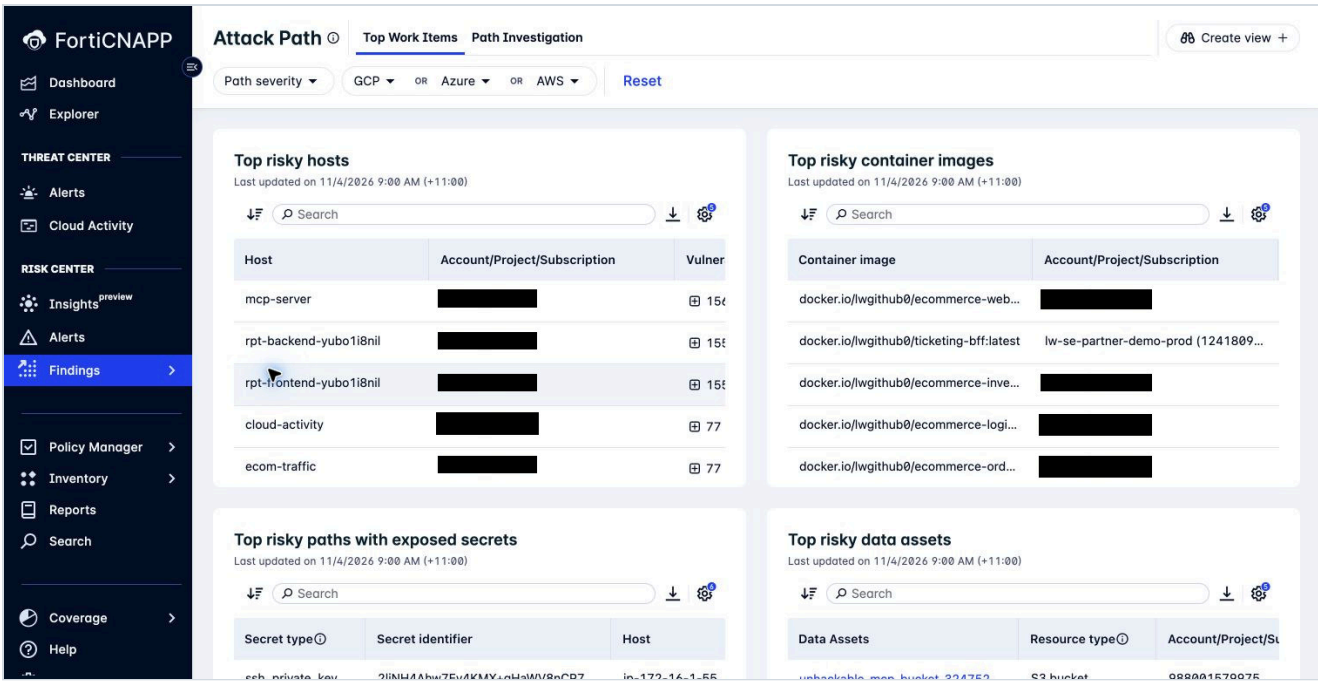
Step 1: Compare the two alert views

1. Open **Risk Center > Alerts**.
2. Open an available risk alert.
3. Identify the resource and condition.
4. Compare its question with the composite alert from Lab 2.

Checkpoint: Record the activity described by your threat alert and the condition described by your risk alert.

Step 2: Find the top work items

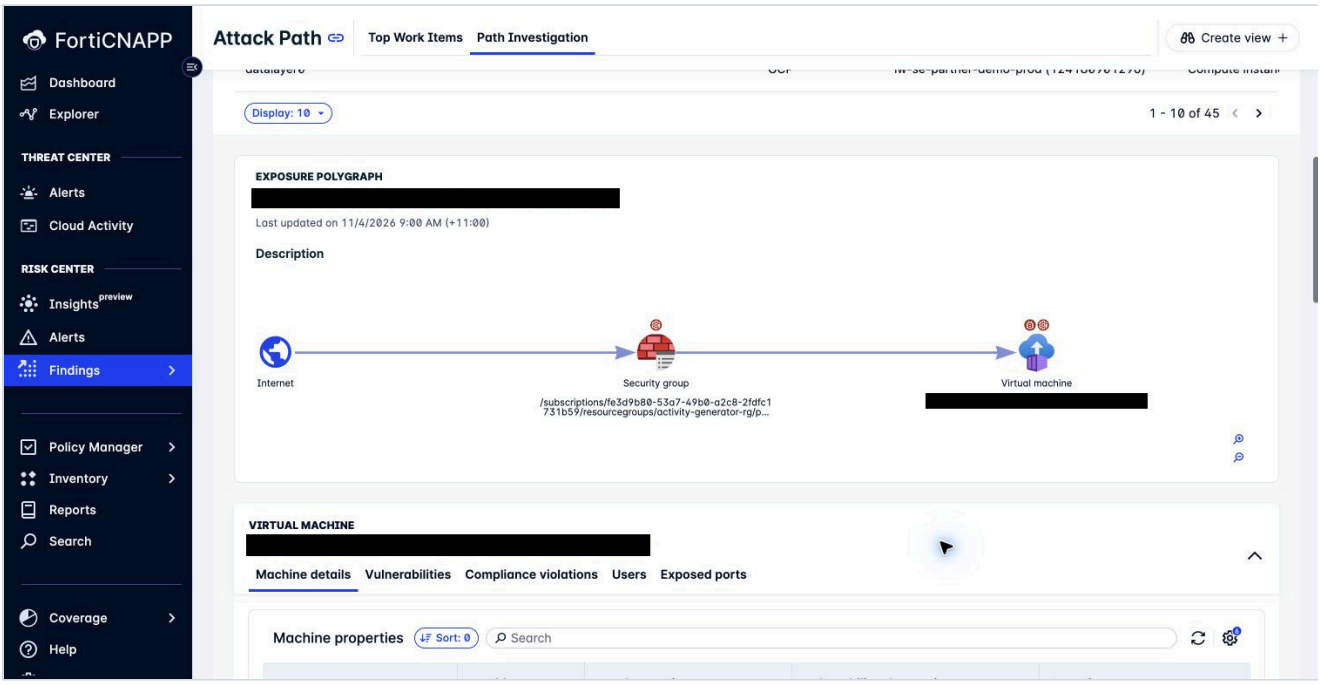
1. Open **Risk Center > Findings > Attack Path**.
2. Select **Top Work Items**.
3. Examine **Top risky hosts**.
4. Examine **Top risky paths with exposed secrets**.
5. Identify the resource that you would investigate first.



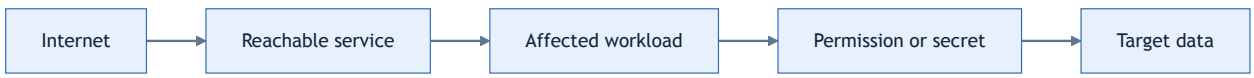
Checkpoint: Name your first work item. Cite one exposure condition and one possible impact from its details.

Step 3: Trace an exposure path

1. Select **Path Investigation**.
2. Look for a path involving your incident host. If none appears, select an AWS or Azure exposure example.
3. Scroll to the **EXPOSURE POLYGRAPH**.
4. Trace the relationships from the entry point to the target.
5. Inspect the available resource details, vulnerabilities, or compliance violations.



Checkpoint: Name an entry point, a target, and one relationship that permits the path.



Example exposure path.

Step 4: Propose a path interruption

1. Select one relationship that you could change.
2. Explain how that change removes or reduces exposure.
3. Identify the service owner who must assess the operational effect.
4. Record how you would verify the change.

Checkpoint: Name the relationship to change, the exposure it removes, and the service check needed before the change.

Do not change cloud resources during this lab.

Next: Lab 6: Decide what to fix first.

Lab 6: Decide what to fix first

Environment: FORTIDEMO-2026-04. **Access:** Read-only.

Concepts

A long list of vulnerabilities does not tell you where to start. Exposure, identity permissions, and available fixes help you choose work that reduces risk. A reachable host with valuable permissions can need a different response from an isolated host with the same vulnerable package.

Objectives

- Justify a remediation priority using exposure, permissions, and available fixes.
- Identify where a source-code check could find a problem before deployment.

Prerequisites

Complete Lab 5. Keep one attack path resource in your worksheet.

Lab steps

Step 1: Examine an identity

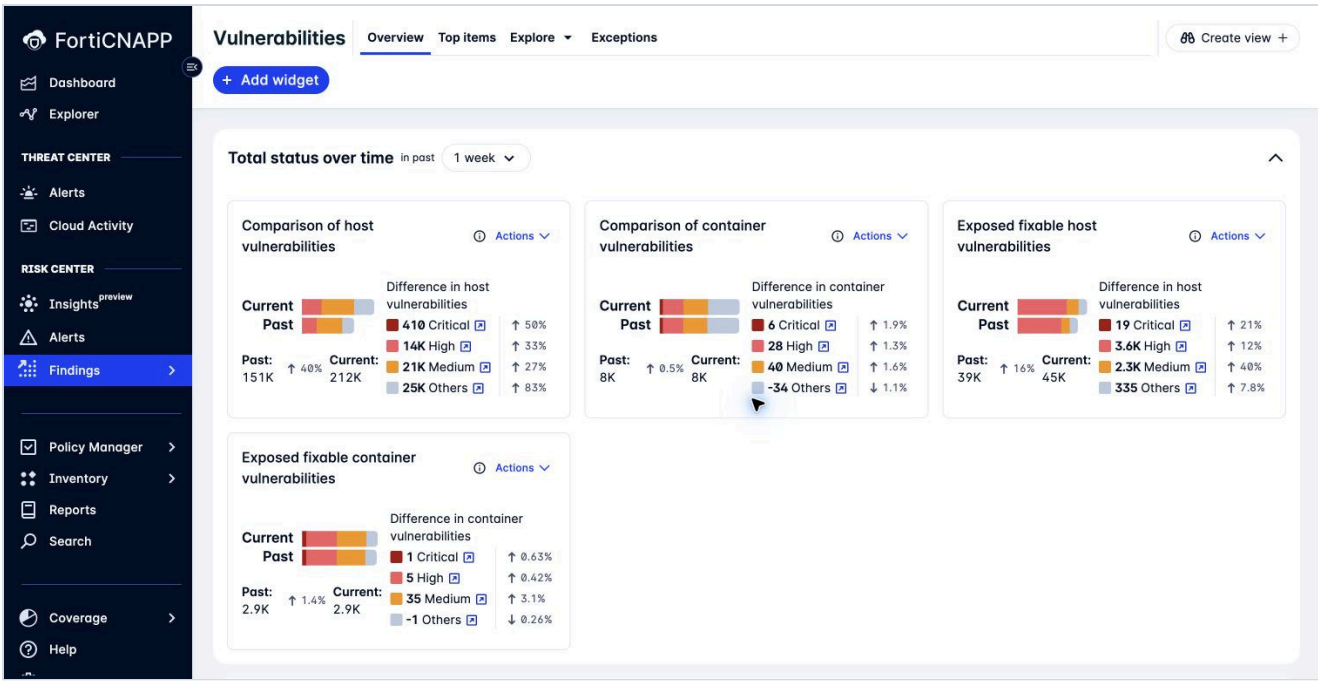
1. Open **Risk Center > Findings > Identities**.
2. Open **Top Identity Risks**.
3. Select an identity from your attack path if available. Otherwise, use another listed identity.
4. Read its risks and available usage information.
5. Compare its permissions with the work it needs to perform.

Checkpoint: Record one permission risk and its usage evidence. State what you would ask the identity owner before changing access.

> **What matters:** An old or unused identity still needs an owner check. Usage history alone does not prove that removing access is safe.

Step 2: Examine a vulnerability

1. Open **Risk Center > Findings > Vulnerabilities**.
2. Open **Top items**.
3. Compare **Top vulnerabilities by impacted hosts** with **Top fixable packages**.
4. Open one finding.
5. Identify the affected package and available fix information.
6. Check the host or image context.



The screenshot shows the Overview tab. Use Top items for the package comparison in this exercise.

Checkpoint: Record the package, affected resource, and available fix. Use the exposure details to justify its place in your work list.

Step 3: Find the earlier control point

1. Open **Findings > Code Security > Infrastructure (IaC)**.
2. Examine an available assessment.
3. Open **Applications**.
4. Compare a dependency finding with a runtime vulnerability finding.

Checkpoint: Identify which evidence concerns source code and which evidence concerns a running workload.

For code scanning exercises, use the AWS integration workshop.

Next: Lab 7: What do your CIS results tell you?.

Lab 7: What do your CIS results tell you?

Environment: FORTIDEMO-2026-04. Use the sandpit for an unavailable Azure example. **Access:** Read-only.

Concepts

A compliance result is useful only when you know what was assessed. CIS benchmarks group security controls by cloud provider and version. FortiCNAPP assesses supported controls against collected resource data. Review failures and assessment gaps together to decide what needs remediation and what needs more evidence.

Objectives

- Identify the provider, benchmark version, scope, and time behind an assessment.
- Trace a failed control to a resource and distinguish it from an assessment gap.

Prerequisites

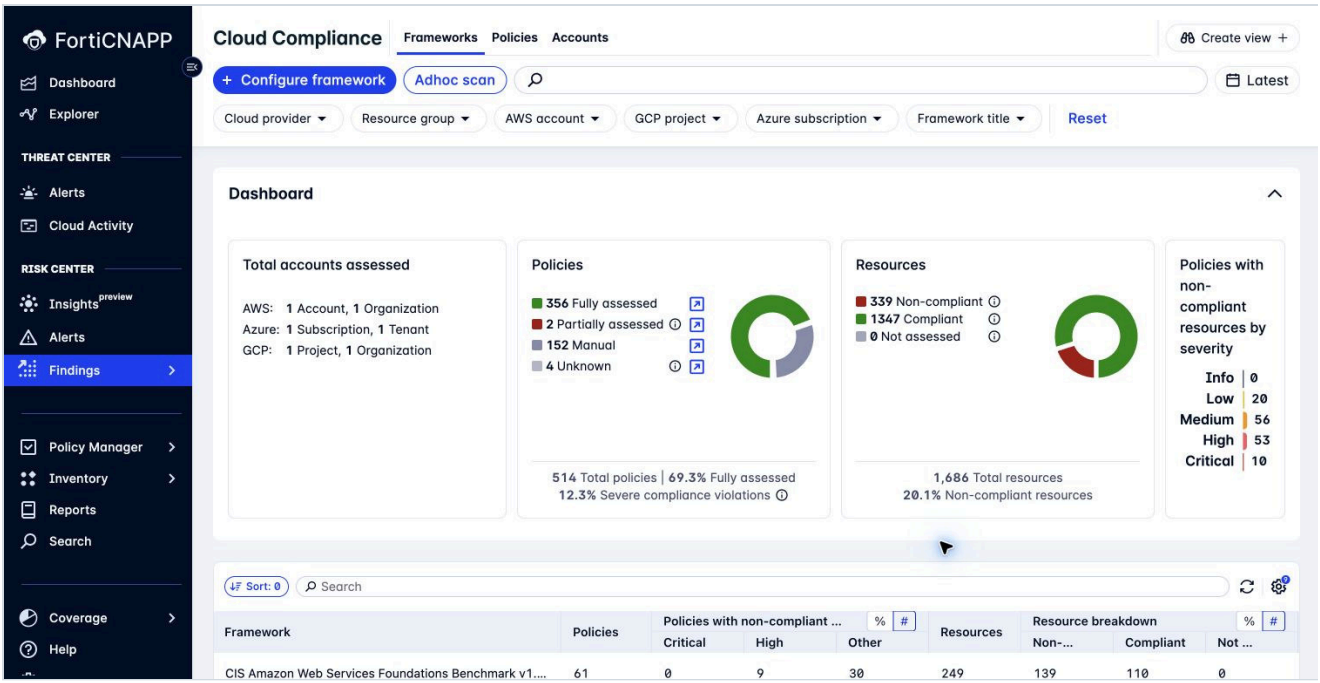
Complete Lab 1.

Lab steps

Step 1: Read a framework result

Use the AWS and Azure scopes assigned for this lab.

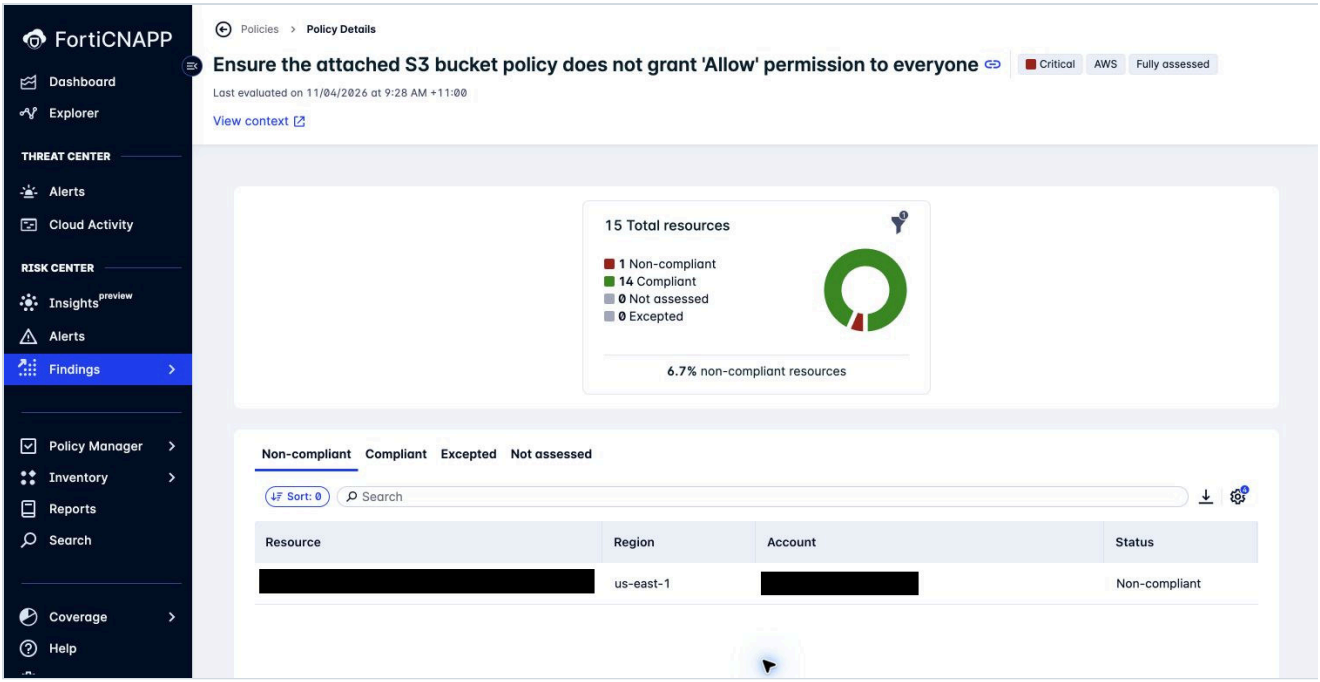
1. Open **Risk Center > Findings > Compliance > Cloud**.
2. Select **Frameworks**.
3. Find an AWS CIS framework.
4. Record its complete title and version.
5. Inspect its assessed resources and policy breakdown.
6. Repeat for an Azure CIS framework that is available.



Checkpoint: Your worksheet has one AWS row and one Azure row. Each row includes the framework title, version, scope, and assessment time. If the demo tenant has no Azure result in the selected scope, use the sandpit for that comparison.

Step 2: Find a failed policy

1. Select **Policies**.
2. Search for `permission to everyone`.
3. Open **Ensure the attached S3 bucket policy does not grant 'Allow' permission to everyone**.
4. Identify a non-compliant resource.
5. Compare **Non-compliant**, **Compliant**, **Excepted**, and **Not assessed**.



Checkpoint: Record the failed resource and the policy condition. Use the resource assessment details to check that they match.

Step 3: Interpret assessment coverage

1. Return to **Frameworks**.
2. Review fully assessed, partially assessed, manual, or unknown policy results that are present.
3. Record one assessment limitation.
4. Explain what evidence a manual control would need.

Checkpoint: Record one failed control and one assessment gap. State whether each needs remediation or more evidence.

> **What matters:** A passing automated assessment does not establish complete CIS certification. Check manual controls, exclusions, coverage, and the exact framework version.

Use the existing assessments for this exercise.

Next: Lab 8: Choose the right policy control.

Lab 8: Choose the right policy control

Environment: FORTINETAPACDEMO. **Access:** Read-only review.

Concepts

Tuning starts with the problem you want to solve: too many matches, the wrong priority, or the wrong recipients. Policy categories evaluate different evidence and expose different controls. A compliance query tests resource configuration. A composite policy combines observations for investigation. Choose a control that addresses the problem without hiding useful evidence.

Objectives

- Compare the available controls across all seven policy categories.
- Propose one tuning change and explain its expected result.

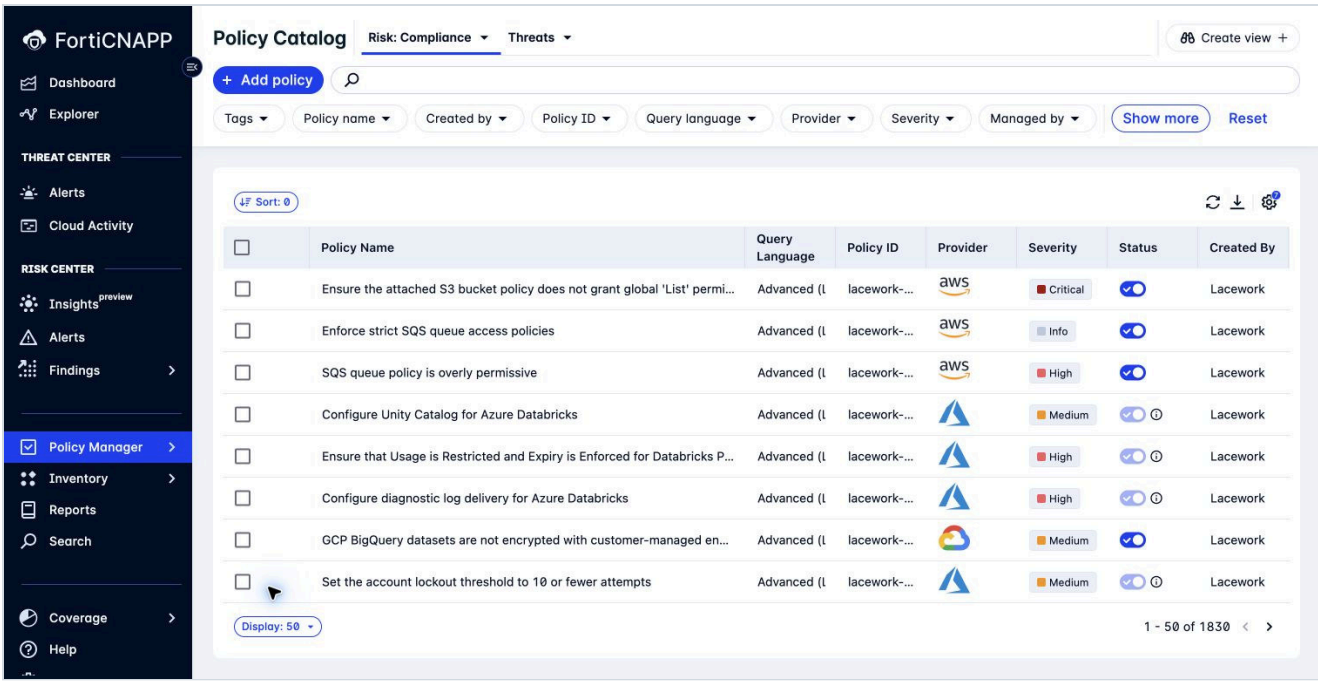
Prerequisites

Complete Labs 2 and 7. Screenshots use the demo tenant. Review sandpit policies without changing them.

Lab steps

Step 1: Map the catalog

1. Confirm **FORTINETAPACDEMO**.
2. Open **Policy Manager > Policies**.
3. Open the **Risk** selector.
4. Locate **Compliance**, **Vuln Build time**, **Vuln Run time**, and **Data** if available.
5. Open the **Threats** selector.
6. Locate **Anomalies**, **Threat Observations**, and **Composite Alerts**.



Use the policy review table in your worksheet. Record one example and its available controls for each category. Mark categories unavailable to your role or tenant.

Checkpoint: Your table has a row for each of the seven categories.

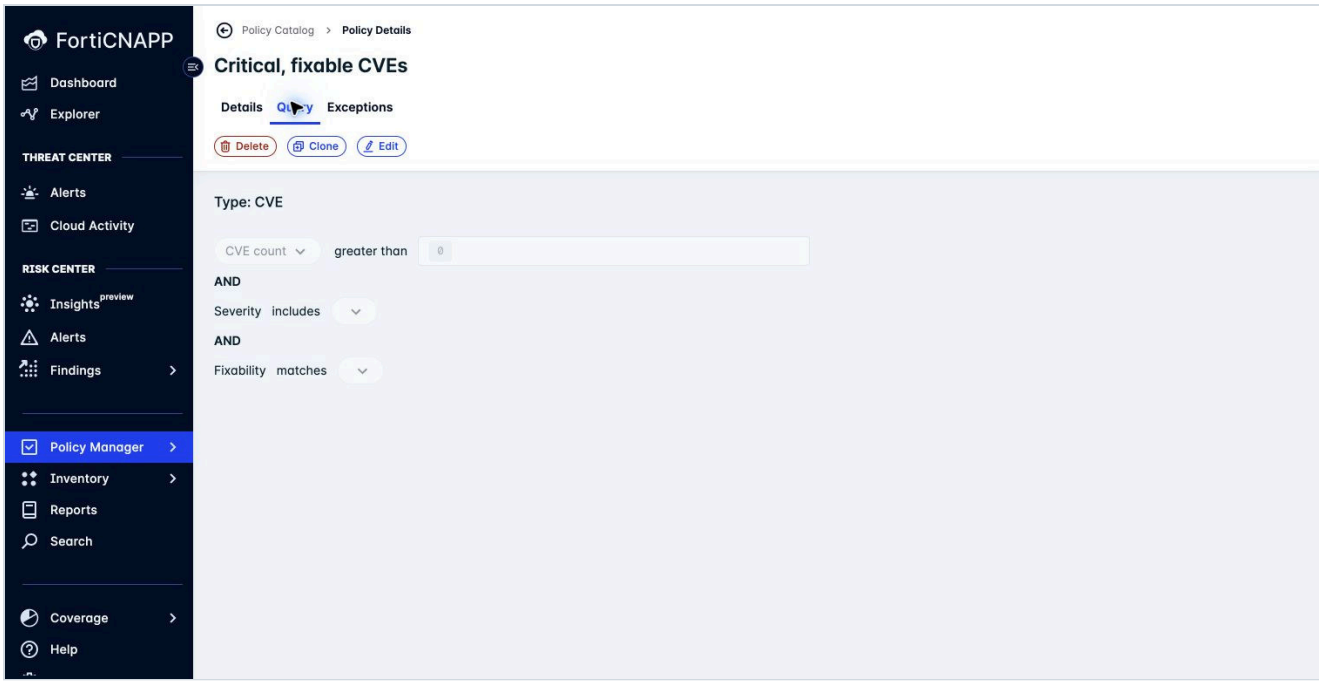
Step 2: Review a compliance policy

1. Open **Risk: Compliance**.
2. Select a policy that you examined in Lab 7.
3. Read its assessed condition, severity, and remediation.
4. Inspect the available editing controls without saving.

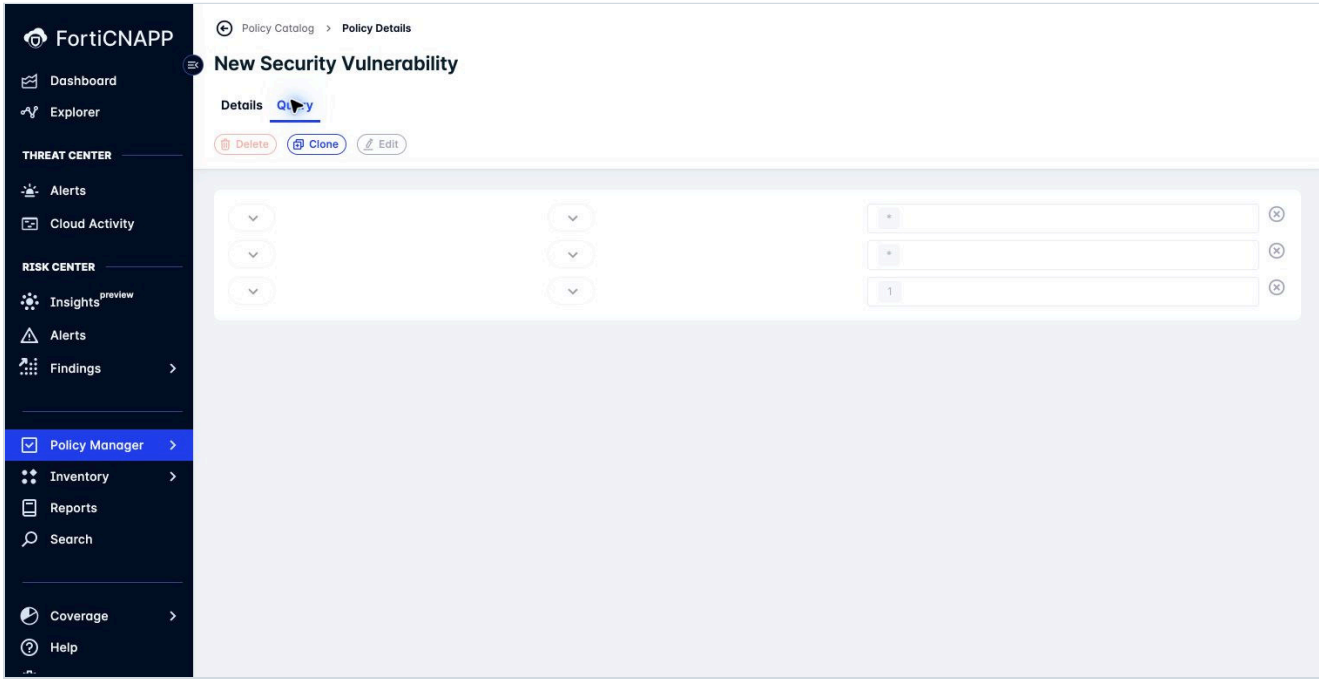
Checkpoint: Record the condition that the policy checks and one available tuning control.

Step 3: Compare vulnerability policies

1. Open **Vuln Build time**.
2. Select **Critical, fixable CVEs**.
3. Read **Action on failure** and the container registry scope in **Details**.
4. Open **Query**.
5. Record the CVE count, severity, and fixability criteria.



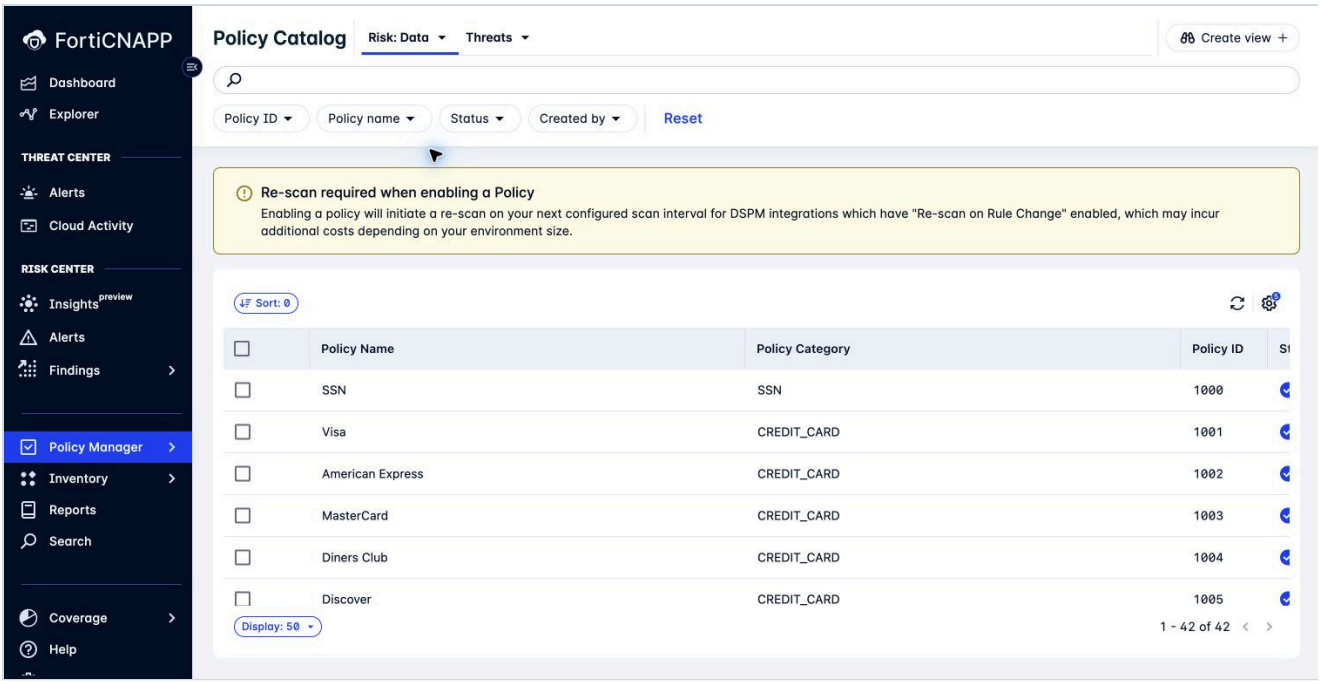
6. Open **Vuln Run time**.
7. Select **New Security Vulnerability**.
8. Compare its **Details** and **Query** tabs with the build-time policy.
9. Record its **Frequency** and query conditions.



Checkpoint: Name one criterion from each policy. Identify whether **Clone** is available for each.

Step 4: Review a Data policy

1. Open **Data** if available.
2. Select one policy.
3. Identify its data security condition and affected assets.
4. Record the available controls without changing them.

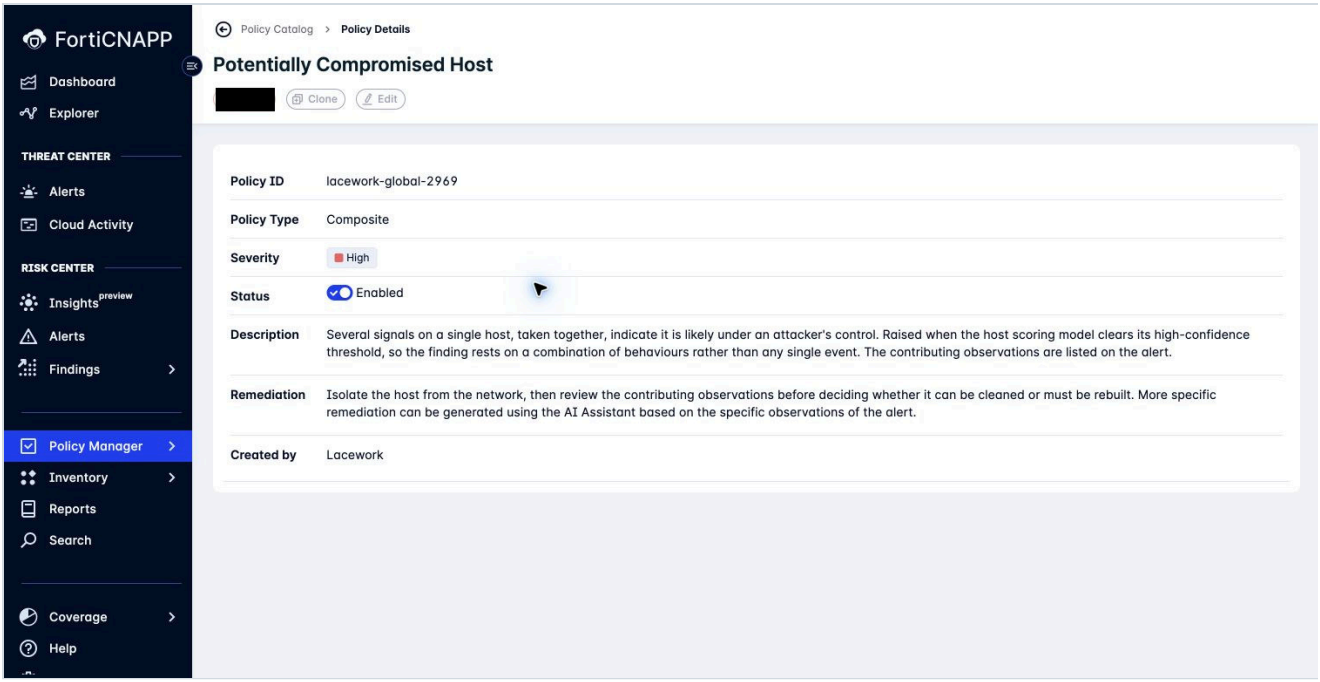


Enabling a Data policy can initiate a rescan for configured DSPM integrations. Keep it unchanged during this review.

Checkpoint: Your Data row identifies an assessed condition or records that the category is unavailable.

Step 5: Read a composite policy

1. Open the **Threats** selector.
2. Select **Composite Alerts**.
3. Open **Potentially Compromised Host**.
4. Read its description and contributing-observation model.
5. Inspect **Edit** and **Clone**.

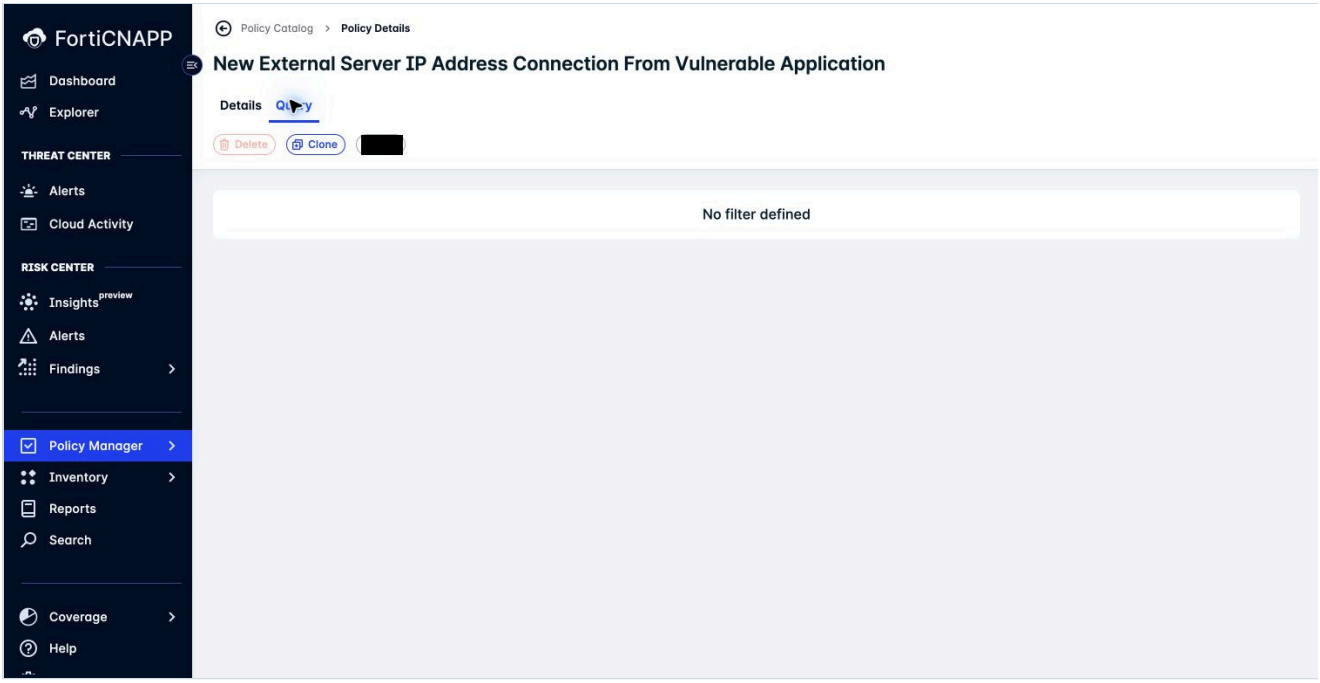


The managed example has disabled Edit and Clone controls.

Checkpoint: Connect the policy description to one observation from your Lab 2 alert. Record whether the editing controls are available in your tenant.

Step 6: Review an anomaly policy

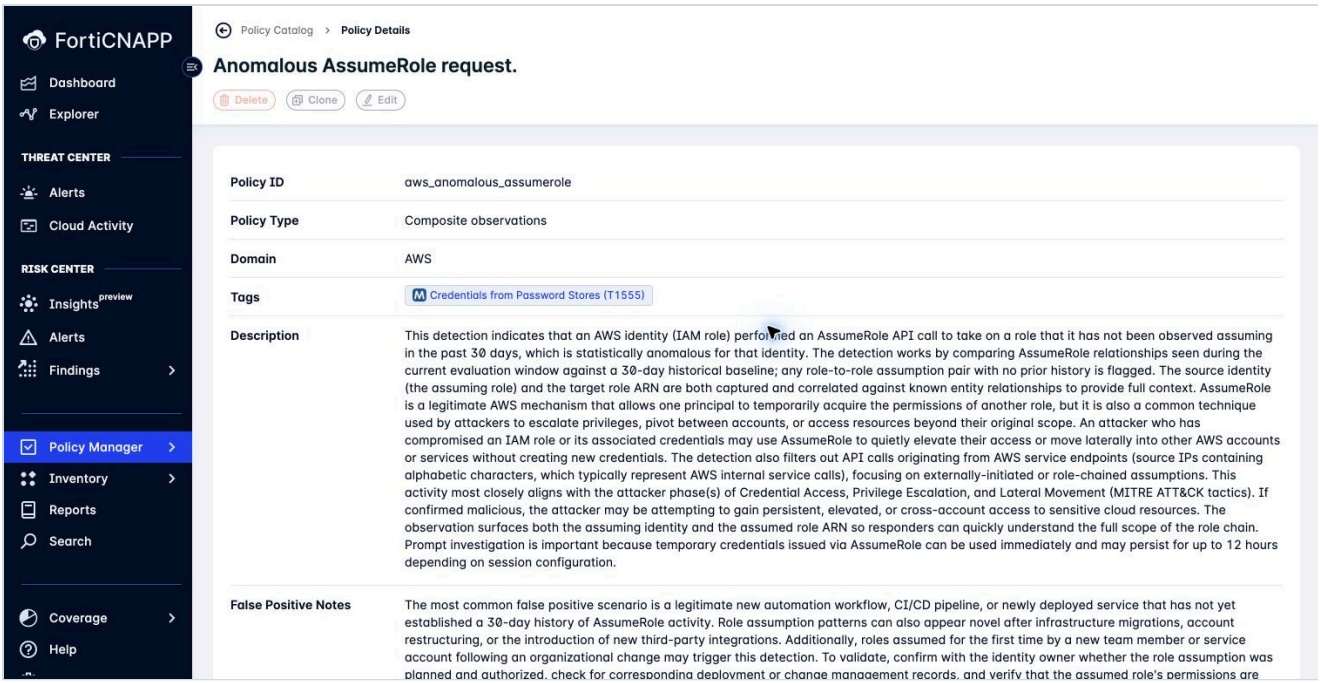
1. Open **Anomalies**.
2. Select **New External Server IP Address Connection From Vulnerable Application**.
3. Read its frequency, severity, domain, and type in **Details**.
4. Open **Query**.
5. Check whether **Clone** is available.



Checkpoint: Your row records **No filter defined** for this example and the state of **Clone**.

Step 7: Read a threat observation definition

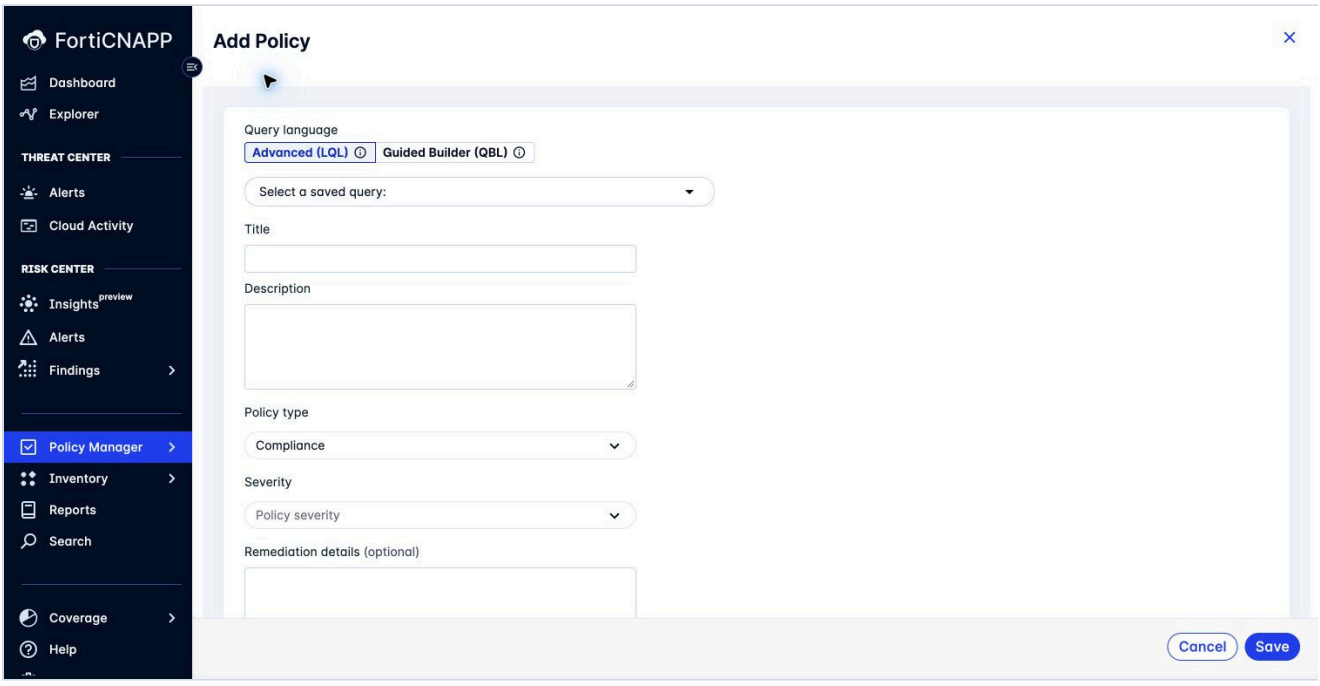
1. Open **Threat Observations**.
2. Select **Anomalous AssumeRole request.** if available. Otherwise, select another definition.
3. Read its description, false positive notes, and triage guidance.
4. Inspect **Edit** and **Clone** without changing the definition.



Checkpoint: Record one triage check and whether the definition can be edited or cloned.

Step 8: Inspect custom policy choices

1. Return to **Risk: Compliance**.
2. Click **Add policy**.
3. Compare **Advanced (LQL)** with **Guided Builder (QBL)**.
4. Inspect **Policy type**.
5. Close the form without saving.



LQL is the query language used in the advanced editor. QBL uses the guided query builder. The LQL path offers Compliance and Violation. The QBL path fixes the policy type to Compliance. A new QBL policy creates a corresponding Risk Insight within about six hours.

Checkpoint: Record the policy types available for each authoring path.

Step 9: Choose a tuning action

Choose one policy from your table. Propose a change using the questions below.

Action	Question to answer first
Change severity	Does business impact justify a different priority?
Change query criteria	Which resources should match, and which should not?
Add an exception	What exact scope and expiry are justified?
Change alert routing	Which team needs this alert?
Disable a policy	What detection or assessment coverage would be lost?

> **What matters:** Choose a control that addresses the problem. Changing severity changes priority; it does not change which resources match.

Checkpoint: Your proposal names an available control, its scope, and the expected result. Include one check for evidence that the change could hide. Keep existing policies unchanged. You will create your own policy in Lab 9.

Next: Lab 9: Turn a security question into a policy.

Lab 9: Turn a security question into a policy

Environment: FORTINETAPACDEMO. **Access:** Read/write.

Concepts

Your team can need a check tailored to its own resources or operating requirements. A query selects data that matches a condition. A policy uses that query to define an assessment or detection. Test the query first. A disabled policy lets you review the result and configuration before activation.

Objectives

- Save a guided query and test an LQL condition against collected EC2 data.
- Create a disabled policy, change one setting, and verify the saved result.

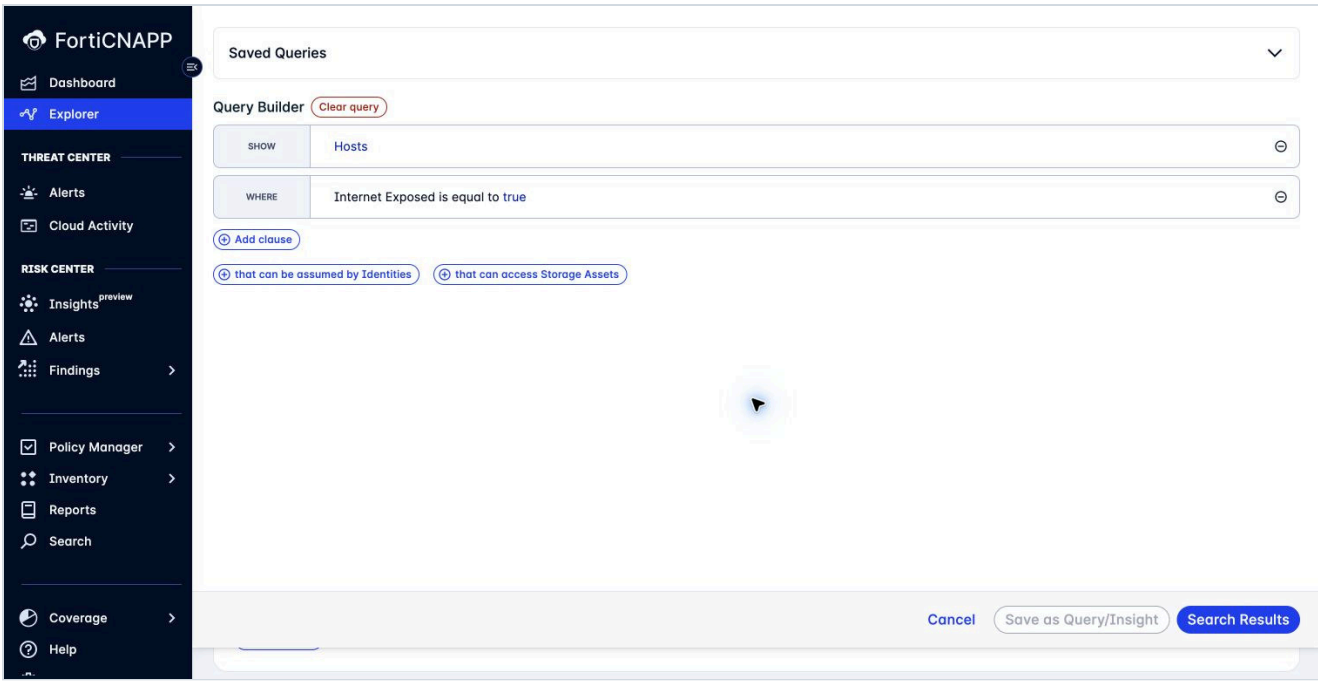
Prerequisites

Complete Lab 8. Screenshots show demo forms. Save objects only in the sandpit.

Lab steps

Step 1: Build a QBL query

1. Confirm **FORTINETAPACDEMO**.
2. Open **Policy Manager > Queries**.
3. Click **Add Query**.
4. Select **Add Explorer Query**.
5. Select **Hosts** as the return type.
6. Click **Add clause** to confirm the return type.
7. Add **Internet Exposed** with the value **True**.
8. Click **Search Results**.
9. Inspect one matching resource.



Checkpoint: Your query contains **Internet Exposed = True**. Open one returned resource and identify the exposure relationship in its graph. If the result is empty, inspect data coverage before concluding that no exposure exists.

Step 2: Save the query

Choose a unique participant ID, such as `ab04`. Replace `YOURID` in workshop names with this ID. For example: `Workshop-ab04-exposed-hosts`. Record each saved object's name and ID under **Created objects** in your worksheet. Use the same participant ID in later labs.

1. Click **Save as Query/Insight**.
2. Set the name to `Workshop-YOURID-exposed-hosts`.
3. Record the generated **Query ID**.
4. Add a description of the intended result.
5. Click **Save Query Only**.
6. Find the query in **Policy Manager > Queries**.

Checkpoint: Your saved query has a unique name and recorded ID. Saving a query does not by itself enable a detection policy.

Step 3: Prepare the sandpit CLI profile

Install `lacework` and `jq` using the CLI reference. Download this repository's files to your computer. Open a terminal in the repository folder. Use the API key supplied for your sandpit account.

Run these commands:

```
lacework version
jq --version
```

Both commands must print a version. The examples use bash or zsh. Replace the key file path with your private path:

```
lacework configure --profile fortinetapacdemo \
  --account partner-demo --subaccount fortinetapacdemo \
  --json_file /PRIVATE/PATH/sandpit-key.json --noninteractive

lacework configure show account --profile fortinetapacdemo
lacework configure show subaccount --profile fortinetapacdemo
```

Checkpoint: The account is `partner-demo`. The subaccount is `fortinetapacdemo`.

> **Caution**
>
> Keep key files and CLI configuration private. Do not paste them into chat, tickets, or Git.

Step 4: Test an LQL condition

Your task is to find EC2 instances that do not require IMDSv2 tokens. Test the condition before saving a disabled policy.

The example finds EC2 configuration where `MetadataOptions.HttpTokens` is not `required`. It checks one IMDSv2 setting, not a complete CIS control.

From the repository root, run:

```
set -o pipefail
lacework query run -f examples/ec2-imdsv2.json \
  --start -7d --end now --limit 5 \
  --profile fortinetapacdemo --account partner-demo \
  --subaccount fortinetapacdemo --json --noninteractive \
  | jq '{sampleRows:length,fields:((.[0] // {}) | keys)}'
```

Checkpoint: The command succeeds. The result reports sample rows and their field names. The five-row limit gives a sample, not a tenant-wide count.

> **Hint:** The seven-day window can include older configuration snapshots. If the result is empty, compare the window with the resource collection time in Explorer.

Step 5: Create a disabled LQL policy

1. Copy `examples/ec2-imdsv2.json` to a private working folder.
2. Replace `REPLACE_ID` in `queryId` with your participant ID.
3. Create the query. Replace `YOUR_QUERY_FILE` with your file path:

```
lacework query create -f YOUR_QUERY_FILE \
  --profile fortinetapacdemo --account partner-demo \
  --subaccount fortinetapacdemo --json --noninteractive
```

4. Record the returned query ID.
5. Prepare a policy file using this shape. Set `queryId` to the exact ID you recorded:

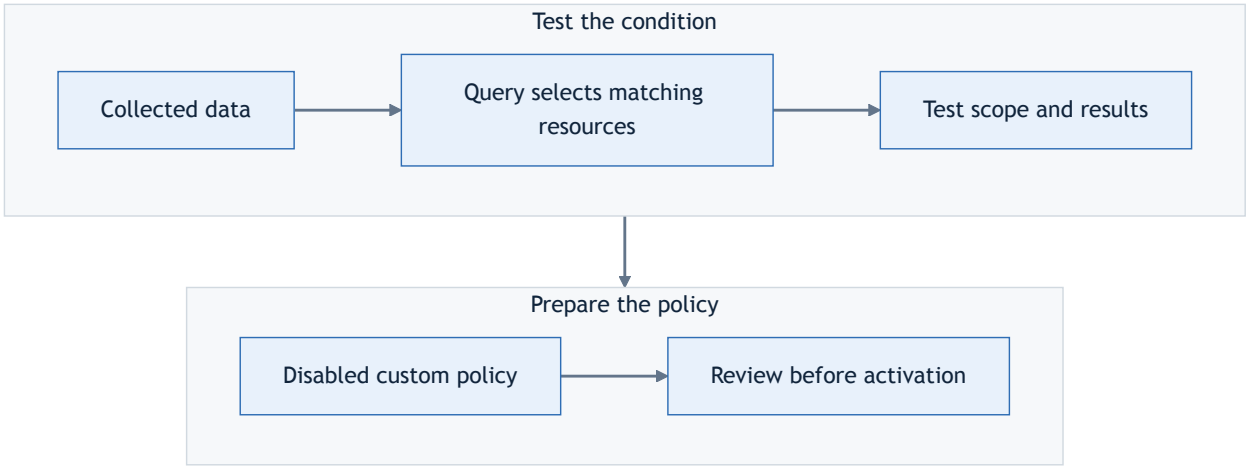
```
title: Workshop-YOURID-IMDSv2
enabled: false
policyType: Violation
alertEnabled: false
queryId: Workshop_IMDSv2_YOURID
severity: high
description: Identify collected EC2 instances that do not require IMDSv2 tokens.
remediation: Review application requirements before requiring IMDSv2 tokens.
```

6. Create the policy. Replace `YOUR_POLICY_FILE` with your file path:

```
lacework policy create -f YOUR_POLICY_FILE \
  --profile fortinetapacdemo --account partner-demo \
  --subaccount fortinetapacdemo --json --noninteractive
```

7. Record the returned policy ID.
8. Find the policy in **Policy Manager > Policies**.
9. Confirm that it remains disabled.

Checkpoint: The saved policy references your query. `enabled` and `alertEnabled` are both false.



Step 6: Review a tuning change

Open your own policy. Change its description or severity using the available editor. Keep it disabled. Read the saved value back.

Checkpoint: Read the edited field back from your policy. It matches your change. The policy remains disabled.

Do not enable the policy during this exercise. Activation requires a separate scope and routing review. Remove both queries and the policy in Lab 16.

Next: Lab 10: Build a compliance framework for your review.

Lab 10: Build a compliance framework for your review

Environment: FORTINETAPACDEMO. **Access:** Read/write.

Concepts

A specific review can need a different selection of controls from a complete benchmark. A framework groups compliance policies into sections for assessment and reporting. A custom framework lets you choose that selection. If you change a CIS framework, present the result as a custom control set.

Objectives

- Create a custom framework with a policy selection that matches your review purpose.
- Verify the saved selection and recognise results that still await evaluation.

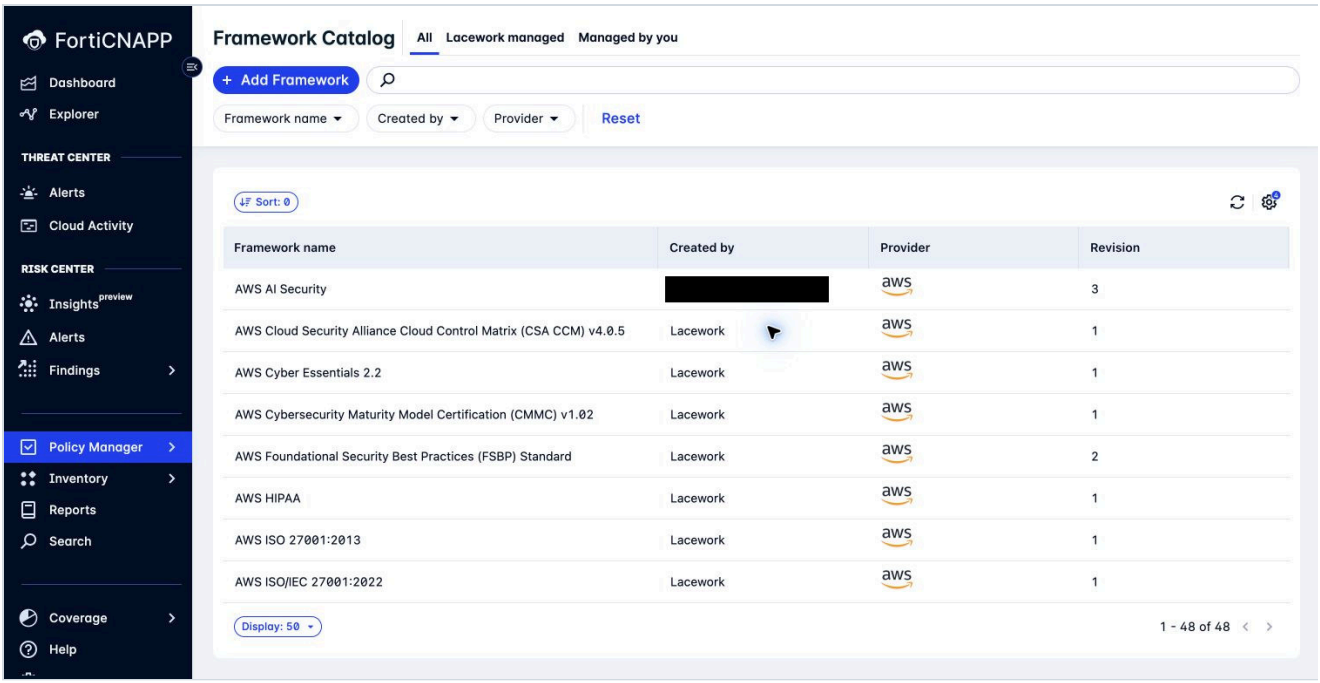
Prerequisites

Complete Lab 7. Use the AWS CIS framework that you examined in Lab 7. Screenshots show the demo tenant. Save only in the sandpit.

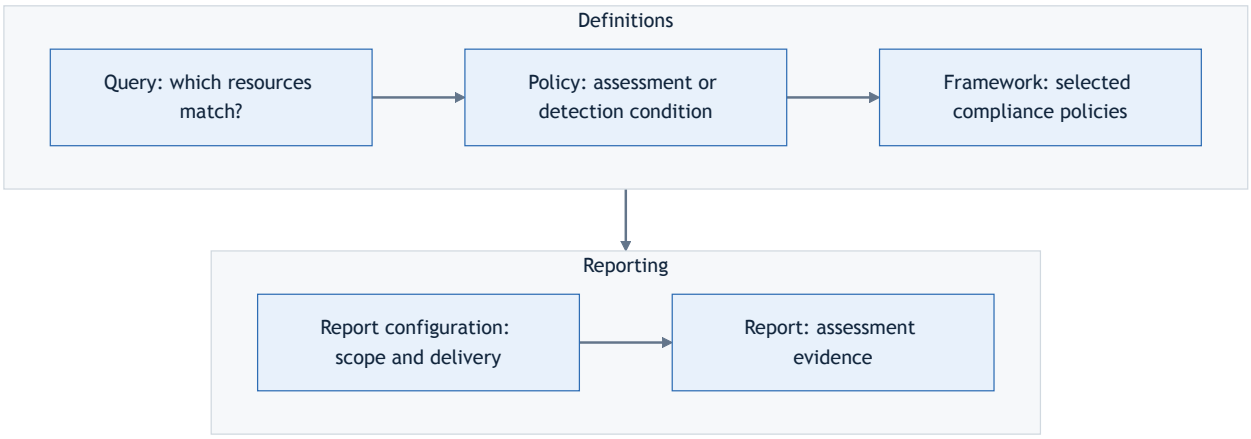
Lab steps

Step 1: Inspect the framework catalog

1. Confirm **FORTINETAPACDEMO**.
2. Open **Policy Manager > Frameworks**.
3. Compare **Lacework managed** with **Managed by you**.
4. Open a built-in framework.
5. Inspect its sections and policy mappings.



Checkpoint: Name one section and a policy mapped to it. The policy defines the check. The framework groups that check with others.



Compliance reporting workflow.

Step 2: Create a custom framework

Use your participant ID from Lab 9. If you skipped that lab, choose a unique ID such as `ab04`. Replace `YOURID` in the name below. Record the saved framework's ID in your worksheet.

1. Return to the catalog.
2. Click **Add Framework**.
3. Select the AWS CIS framework from Lab 7.
4. Continue to the framework editor.
5. Set a unique name: `Workshop-YOURID-AWS-review`.
6. Select **Networking** in the copied framework.
7. Click **Add / Edit policies**.
8. Select **Selected policies**.
9. Choose one policy to omit from your custom review.

10. Record why it is outside your review scope.
11. Clear that policy.
12. Click **Update section**.
13. Save your custom framework.

Checkpoint: Your framework appears under **Managed by you**. Reopen its edited section. The omitted policy is absent. The source framework is unchanged.

> **What matters:** A modified CIS-based framework is a custom control set. Do not present it as the complete, unchanged CIS benchmark.

Step 3: Review revision and evaluation state

1. Reopen your framework.
2. Record its revision and policy selection.
3. Check for **Pending changes**.
4. Explain which assessment results may still reflect the earlier configuration.

Checkpoint: Your worksheet records the framework name, provider, revision, and reporting purpose.

Framework changes can take up to 24 hours to appear in assessment results. Continue with the existing assessment while the changes are evaluated. Keep this framework for cleanup. Lab 11 uses the built-in CIS frameworks for its main reports.

Next: Lab 11: Build AWS and Azure CIS reports your team can use.

Lab 11: Build AWS and Azure CIS reports your team can use

Environment: FORTINETAPACDEMO. **Access:** Read/write.

Concepts

A report must give its readers the right evidence for the right resources. Its configuration selects the framework, scope, assessment statuses, and delivery settings. The generated report contains the assessment evidence for those selections. Separate AWS and Azure configurations keep each provider's benchmark and scope clear.

Objectives

- Create separate AWS and Azure CIS report configurations with the intended scope.
- Check assessment gaps, recipients, and delivery settings before enabling distribution.

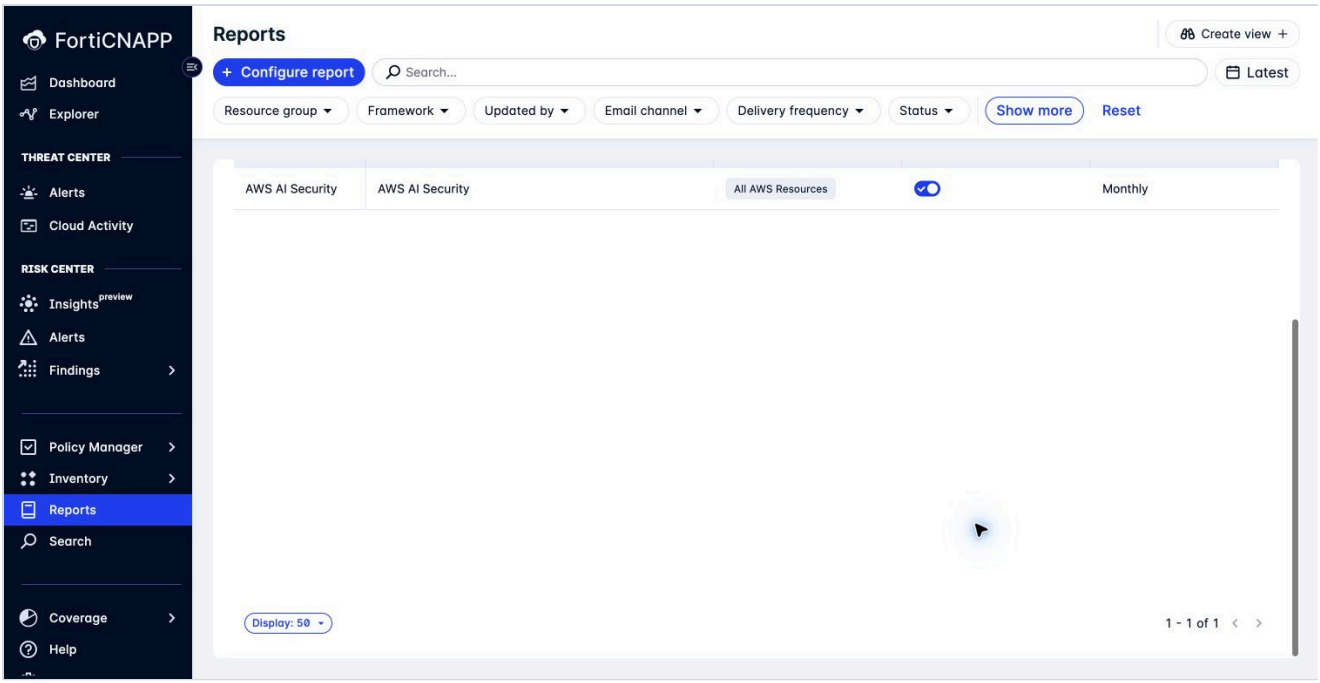
Prerequisites

Complete Labs 7 and 10. Screenshots show demo report controls. Create reports only in the sandpit.

Lab steps

Step 1: Inspect existing configurations

1. Confirm **FORTINETAPACDEMO**.
2. Open **Reports**.
3. Inspect an AWS report configuration.
4. Inspect an Azure report configuration.
5. Record the framework and data scope for each.

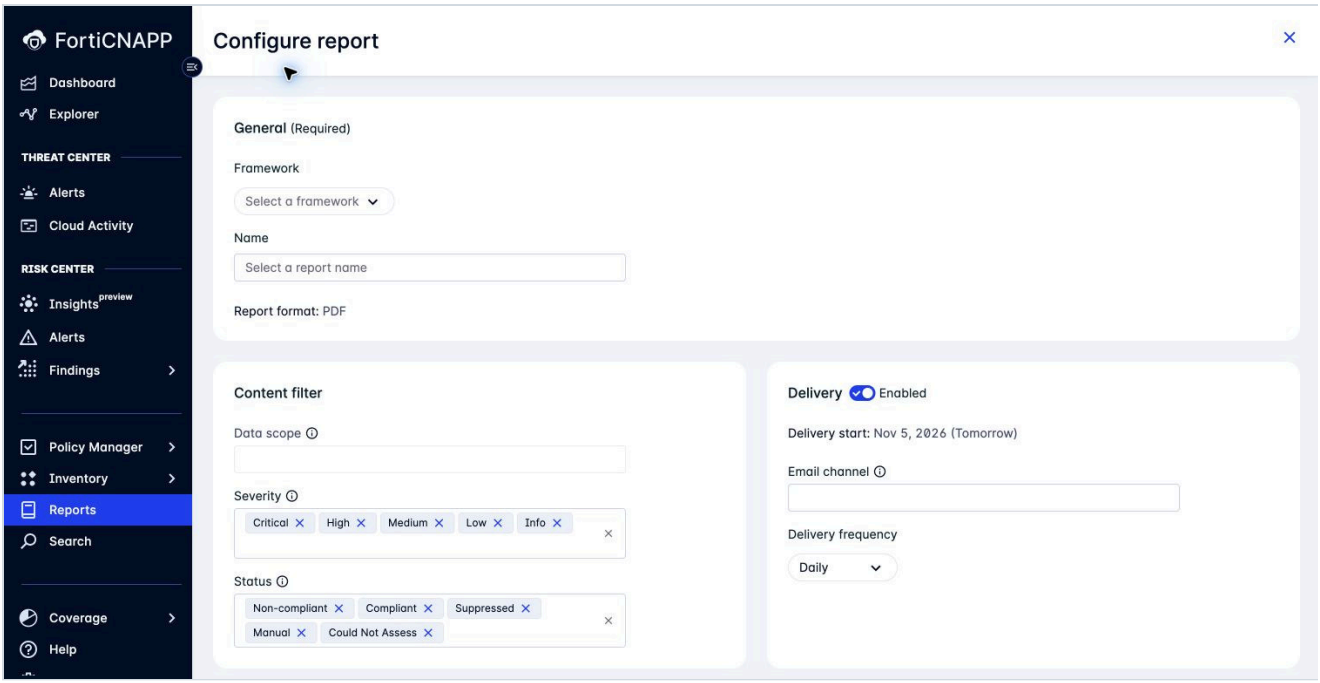


Checkpoint: Record the framework and scope from each configuration. These are instructions for report generation, not assessment results.

Step 2: Configure the AWS report

Use the AWS resource group assigned for this lab. Use your participant ID in the report name.

1. Return to **Reports**.
2. Click **Configure report**.
3. Select the required AWS CIS framework and version.
4. Set **Name** to `Workshop-YOURID-AWS-CIS`.
5. Select the approved AWS **Data scope**.
6. Review **Severity**.
7. Review **Status**.
8. Turn off **Enabled** under **Delivery**.
9. Select the **Email channel** assigned for this lab.
10. Review **Delivery frequency**.
11. Create the configuration.
12. Open the saved configuration.
13. Confirm that **Report configuration** is **Disabled**.
14. Turn off its switch if the saved configuration is enabled.



Checkpoint: The saved configuration has the required AWS framework and scope. Delivery is disabled.

An email channel is required even when delivery is off. Check the saved configuration's state. Do not rely only on the creation form's switch.

> **What matters:** A report can hide assessment gaps if you exclude **Manual** and **Could Not Assess** results.

Step 3: Configure the Azure report

Repeat Step 2 with the required Azure CIS framework. Use the assigned Azure resource group. Use `Workshop-YOURID-Azure-CIS` as the name. Select the approved Azure subscription scope. Keep delivery disabled.

Checkpoint: Reopen both configurations. Compare their provider, version, and scope with your Lab 7 assessment record. Both remain disabled.

Step 4: Review the delivery plan

1. Record the intended recipients and frequency in your worksheet.
2. Compare them with the selected email channel.
3. Identify the owner who approves enabling delivery.
4. Check the available report preview from **Cloud Compliance > Frameworks**.

Checkpoint: Your delivery plan names the recipients, frequency, framework, and resource scope. Use the saved settings to check each item.

Do not enable delivery to a shared email channel during the lab. Keep both configurations for cleanup in Lab 16.

Next: Lab 12: Can you trust the data behind your findings?.

Lab 12: Can you trust the data behind your findings?

Environment: FORTINETAPACDEMO. **Access:** Read-only review.

Concepts

An investigation can miss activity when collection is stale or incomplete. Even a useful alert needs to reach a responsible team. Collection health, alert routing, and report delivery are separate checks. Checking each one helps you find gaps between collected evidence and the people who need it.

Objectives

- Check whether cloud and workload evidence is current.
- Trace notification settings and write a handover for a collection issue or status update.

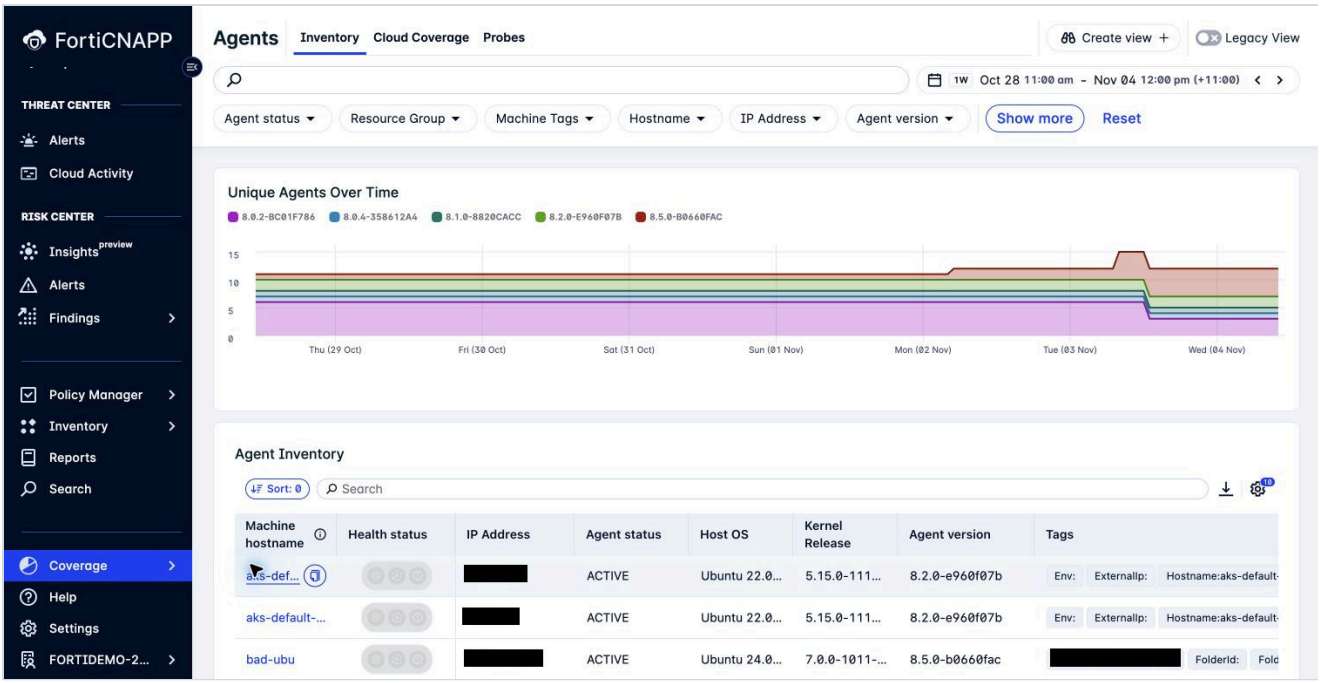
Prerequisites

Complete Lab 11. Do not change shared integrations or channels.

Lab steps

Step 1: Check collection

1. Open **Settings > Cloud accounts**.
2. Inspect the configuration integration for an AWS or Azure scope.
3. Check its state and last successful collection.
4. Inspect the activity log integration separately.
5. Open **Coverage > Agents**.
6. Check agent status and last check-in.
7. Open **Coverage > Agentless**.
8. Compare that coverage with the agent view.



Check collection state in the sandpit.

Checkpoint: Record one collection source and its latest successful update. An enabled integration and a healthy collection state are different checks.

Step 2: Review routing

1. Open **Settings > Channels**.
2. Inspect a channel without changing its recipients.
3. Examine the alert-routing controls available to your role.
4. Trace the intended route for a composite alert.
5. Compare that route with the email channel in your report configuration.

Checkpoint: Record one policy, the alert route you inspected, and the report email channel. Label which defines the condition, routes the alert, and delivers the report.

> **What matters:** Check that alert notifications reach the responsible team.

Step 3: Record an operational issue

Write a short handover for one failed or stale collection. If none is present, use a healthy collection as a status update. Include the integration type, time, observed state, and next check. Keep credentials and unrelated identifiers out of the handover.

Checkpoint: Your handover reports the observed issue without claiming an unverified cause.

Next: Lab 13: Ask the same questions through the CLI.

Lab 13: Ask the same questions through the CLI

Environment: Both tenants. **Access:** Read-only.

Concepts

The console is useful for exploration. Commands let you repeat a check or use its result in a script. The CLI retrieves FortiCNAPP data for a selected account and subaccount. Matching the console's scope and time window helps you check that both tools answer the same question.

Objectives

- Retrieve composite alert and collection summaries through the CLI.
- Compare those summaries with the console using the same scope and time window.

Prerequisites

Complete the CLI setup in Lab 9 Step 3. Run commands from the repository root.

Lab steps

Step 1: Check the tools

```
lacework version
jq --version
```

Checkpoint: Both commands print a version. The examples use bash or zsh. Use an approved equivalent shell for other systems.

Step 2: Add the demo profile

Keep the sandpit profile from Lab 9. Add a separate demo profile. Use the API key supplied for your read-only demo account. Replace the key file path below with your private path.

```
lacework configure --profile fortidemo-2026-04-readonly-use \
  --account partner-demo --subaccount fortidemo-2026-04 \
  --json_file /PRIVATE/PATH/demo-key.json --noninteractive
```

The command saves local credentials. It does not create an API key.

Checkpoint: The configuration command completes without an error. Keep the downloaded key file and local CLI configuration private.

Step 3: Verify the selected scope

```
lacework configure show account --profile fortidemo-2026-04-readonly-use
lacework configure show subaccount --profile fortidemo-2026-04-readonly-use
```

Checkpoint: The account is `partner-demo`. The subaccount is `fortidemo-2026-04`. Do not print the complete CLI configuration.

Step 4: Find composite alerts

```
set -o pipefail
sh tools/lw-read.sh demo alert list --start -7d --end now \
  | jq '[.[] | select(.derivedFields.category == "Composite")]
    | group_by(.alertName)
    | map({name:.[0].alertName,count:length,
        severities:([.[].severity] | unique)})'
```

Checkpoint: Compare the titles and period with the console. If the selected alert is older, use the same historical period in both tools. Each alert-list request must use a window of seven days or less.

> **Hint:** A month in the console and seven days in the CLI are different datasets. Match time, category, severity, and status before comparing counts.

Step 5: Summarise integration coverage

```
sh tools/lw-read.sh sandpit cloud-account list \
  | jq 'group_by(.type)
    | map({type:.[0].type,total:length,
        enabled:([.[] | select(.enabled == 1)] | length),
        healthy:([.[] | select(.state.ok == true)] | length)})'
```

Compare `enabled` and `healthy` for one integration type. Use **Settings > Cloud accounts** to check the state of its integrations.

Checkpoint: Record the two counts and one integration state from the console. Use that state to explain whether enabled collection is healthy.

The wrapper accepts selected read commands and fixes the tenant flags.

Next: Lab 14: Get the evidence behind an alert through the API.

Lab 14: Get the evidence behind an alert through the API

Environment: Both tenants. **Access:** Read-only.

Concepts

Automation needs access to evidence, not just a view of it. An API request specifies a method, endpoint, and parameters. Filters and pagination determine how much data the response contains. Check the returned values against your console investigation before using them in another workflow.

Objectives

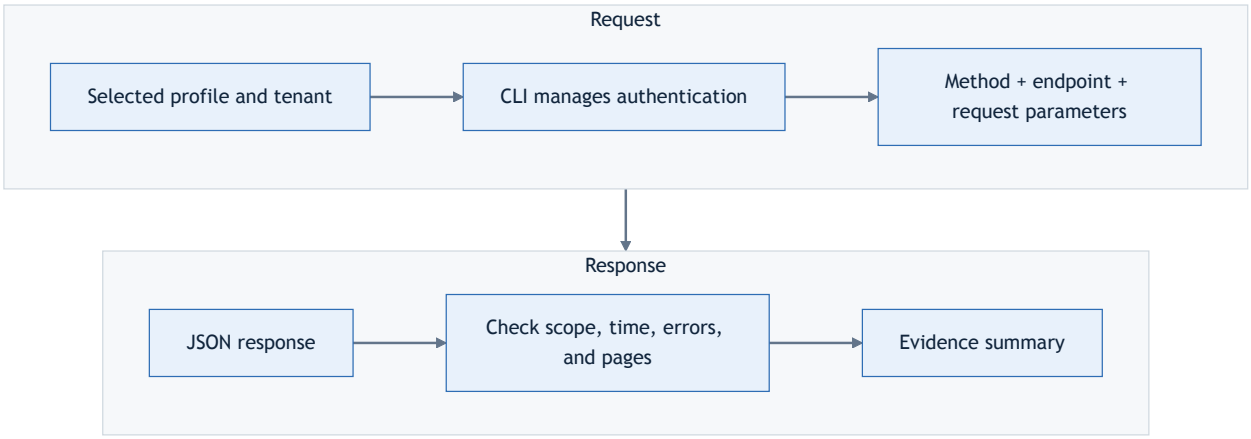
- Retrieve an alert through the API and match its values to console evidence.
- Determine whether a result is complete, partial, or a failed check.

Prerequisites

Complete Lab 13. Keep an alert ID from your composite investigation. Use the API endpoint documentation for request fields.

Lab steps

Step 1: Identify the request parts



The `lacework api` command manages authentication for the selected profile.

Step 2: Compare a CLI list with an API response

```
set -o pipefail
sh tools/lw-read.sh sandpit api get /api/v2/CloudAccounts \
  | jq '{responseFields:keys,returnedRows:(.data | length)}'
```

Compare the shape with `cloud-account list` from Lab 13. The native list command returns an array. This REST response wraps rows in `data`.

Checkpoint: The API summary reports `responseFields` and `returnedRows`. Identify `data` in the response fields. That is where this response stores its rows.

Step 3: Retrieve one alert

Set the ID from your own investigation:

```
ALERT_ID=REPLACE_WITH_ALERT_ID
ALERT_FILE=$(mktemp)
sh tools/lw-read.sh demo api get \
  "/api/v2/Alerts/${ALERT_ID}?scope=Details" > "$ALERT_FILE"
jq '{responseFields:keys,dataFields:(.data | keys)}' "$ALERT_FILE"
```

Open the response in your private terminal:

```
jq '.data' "$ALERT_FILE" | less
```

In `less`, type `/` followed by an entity value from Lab 2. Press Enter to search. Use `n` for the next match. Press `q` to return to the shell.

1. Find an entity or activity value that also appears in the console.
2. Record the JSON field name and its matching console value in your private worksheet.
3. Repeat for a second value. Record any evidence you could not find.

Checkpoint: Two console values match the API evidence, or your worksheet identifies the missing evidence. Field names alone are not evidence of the incident.

Alert details can contain identities, addresses, and command lines. Keep the file private. Remove it after the comparison:

```
rm -f "$ALERT_FILE"
```

Step 4: Understand search requests

A search endpoint can use `POST` without changing tenant configuration. Check the endpoint documentation to determine its effect.

Run this read-only search in the sandpit:

```
REQUEST_FILE=$(mktemp)
jq -n '{timeFilter:{
  startTime:(now-86400 | floor | todate),
  endTime:(now | floor | todate)}}' > "$REQUEST_FILE"

lacework api post /api/v2/Alerts/search --file "$REQUEST_FILE" \
  --profile fortinetapacdemo --account partner-demo \
  --subaccount fortinetapacdemo --json --noninteractive \
  | jq 'if . == null then {returnedRows:0,hasMore:false}
    else {returnedRows:(.data | length),
      hasMore:(.paging.urls.nextPage != null)} end'

rm -f "$REQUEST_FILE"
```

Check `timeFilter` before you run the request. It selects the last 24 hours.

Checkpoint: The result reports `returnedRows` and `hasMore`. Find the endpoint description that confirms this operation searches alerts.

Step 5: Check completeness

1. Inspect the endpoint's response schema.
2. Check for paging metadata and a next-page link.
3. Retrieve all required pages before calculating a full total.
4. Record the time window and filters with the result.
5. Preserve request failures as failures.

Checkpoint: Record your time window, filters, and `hasMore` value. If another page exists, your current row count is incomplete. A request error is an incomplete check.

> **What matters:** An empty result, a failed request, and a first page are different outcomes. Do not turn any of them into an unsupported claim about tenant security.

Next: Lab 15: Repeat your investigation with the FortiCNAPP agent skill.

Lab 15: Repeat your investigation with the FortiCNAPP agent skill

Environment: Both tenants. **Access:** Read-only.

Concepts

An external coding agent can help collect and summarise evidence through the CLI and API. The FortiCNAPP agent skill gives it instructions for that work. Repeating a known investigation gives you a reference for checking its answer. The skill is separate from the workload agent and console AI Assist.

Objectives

- Use the skill to repeat your composite investigation and check collection coverage.
- Verify its main claims against the console or API evidence.

Prerequisites

Complete Labs 2, 13, and 14. Keep the CLI profiles from Lab 13.

Lab steps

Step 1: Identify the three agents

Component	Purpose
FortiCNAPP workload agent	Collects workload telemetry
FortiCNAPP AI Assist	Provides assistance inside the console
FortiCNAPP agent skill	Supplies investigation instructions to an external coding agent using the CLI and API

Checkpoint: Explain which component collects process telemetry and which component runs CLI/API investigations.

Step 2: Install or inspect the skill

Install an approved AI coding agent if needed. Confirm that your organisation permits the selected tenant data in that runtime. Install Git if needed for the clone command below.

Use the skill repository for current runtime setup. For Codex, the repository documents this installation target:

```
git clone https://github.com/andrewbearsley/forticnapp-lacework-skill.git \
  ~/.agents/skills/forticnapp-lacework
```

If the skill is already installed, inspect its `SKILL.md` instead. Do not clone over an existing installation. Confirm that the selected runtime loads the skill before continuing.

Checkpoint: Identify the installed skill version and its read-only defaults.

Step 3: Repeat the composite investigation

Invoke `$forticnapp-lacework` in Codex. Supply this task with your chosen ID and period:

```
Use profile fortidemo-2026-04-readonly-use, account partner-demo,
and subaccount fortidemo-2026-04.
Use read operations only. Do not change any tenant state.
Investigate composite alert REPLACE_ID for REPLACE_PERIOD.
Confirm the tenant before requesting data.
Summarise observations, entities, and the activity sequence.
Separate observed facts from hypotheses.
State the evidence sources and any incomplete checks.
Reduce output before displaying it. Keep secrets and unrelated identifiers private.
```

Checkpoint: Compare the result with your Lab 2 worksheet. Check at least two claims against the console or API evidence.

Step 4: Check sandpit coverage

```
Use profile fortinetapacdemo, account partner-demo,
and subaccount fortinetapacdemo.
Use read operations only.
Check cloud integration status and workload agent coverage.
Report counts, stale collection, and failures.
Distinguish data collection health from security findings.
Do not change policies, alerts, integrations, or reports.
```

Checkpoint: A failed integration is reported as an issue. A request failure remains an incomplete check.

Step 5: Review the answer

Ask the agent which source supports its main conclusion. Compare the requested time period with the period it actually queried. Check whether it handled all required pages. Correct any unsupported statement in your worksheet.

Checkpoint: Name one useful result from the agent and one claim that you checked yourself. Cite the console or API evidence used for that check.

Next: Lab 16: Present your findings and clean up.

Lab 16: Present your findings and clean up

Environment: Demo for investigation; sandpit for cleanup.

Concepts

An investigation is useful when another person can understand the evidence and act on your recommendation. Keep detected activity, possible exposure, and compliance results distinct in your explanation. Removing your saved lab objects leaves the shared sandpit ready for the next exercise.

Objectives

- Present an investigation with supported conclusions and a proposed response.
- Explain your risk and compliance evidence.
- Remove your saved objects and verify that they are gone.

Prerequisites

Keep your evidence worksheet and the IDs of every object that you created.

Lab steps

Step 1: Present the investigation

1. Confirm **FORTIDEMO-2026-04**.
2. Present the composite alert and its activity window.
3. Explain two related observations.
4. State the affected entities and evidence sources.
5. Separate confirmed facts from hypotheses.
6. Propose a response and a verification method.

Checkpoint: Another participant can trace each main conclusion to its evidence. Do not change the demo alert's status or comments.

Extension: Investigate **Potentially Compromised AWS Credentials** if available. Apply the same method to identities, API activity, and related resources.

Step 2: Explain the risk and compliance evidence

1. Explain an exposure path and a possible interruption.
2. State why that path does not prove exploitation.
3. Identify the CIS framework provider and version.
4. Explain one failed control and one assessment gap.
5. Show the scope of each sandpit report configuration.

Checkpoint: Your explanation keeps incident evidence, exposure, and compliance assessment distinct.

Step 3: Remove your report configurations

1. Switch to **FORTINETAPACDEMO**.
2. Open **Reports**.
3. Find each configuration from your created-object record.
4. Confirm its exact name and framework.
5. Remove only your workshop configurations using the available action.
6. Verify that they no longer appear.

Checkpoint: No workshop report configuration remains for your participant ID.

Step 4: Remove your policies, framework, and queries

1. Open **Policy Manager > Policies**.
2. Remove your disabled workshop policy by its recorded ID.
3. Open **Policy Manager > Frameworks**.
4. Remove your custom workshop framework.
5. Open **Policy Manager > Queries**.
6. Remove your QBL and LQL workshop queries.
7. Verify that each recorded ID is absent.

Remove dependent objects first. Do not remove a query that an existing shared policy uses. Never select all objects for cleanup.

Checkpoint: Your created-object record shows the removal of every object you created.

Step 5: Close the session

Close private exports and credential files. Remove temporary credential files that you no longer need. Do not revoke a shared key or remove another participant's local profile.

Checkpoint: Your workshop objects and temporary credential files are removed.

Return to the workshop overview.

Investigation worksheet

Keep your completed worksheet in approved private storage.

Field	Your evidence
Participant ID	
Tenant	
Alert title and ID	
Time range and timezone	
Composite observations	
Primary entities	
Confirmed facts	
Hypotheses that need checking	
Proposed response	
Attack path entry point	
Target resource	
Proposed path interruption	
CIS provider and benchmark version	
Failed control and affected resource	
Manual or unassessed controls	
Report scope and delivery settings	
CLI/API source and pagination status	
Skill conclusion checked against evidence	

CIS assessments

Provider	Framework and version	Resource scope	Assessment time	Gap to investigate
AWS				
Azure				

Policy review

Category	Example policy	Available controls	Review finding
Compliance			
Vuln Build time			
Vuln Run time			
Data			
Anomalies			
Threat Observations			
Composite Alerts			

Created objects

Tenant	Object type	Name	ID	Removed
FORTINETAPACDEMO				

Completion

Explain the incident evidence to another participant. Explain the difference between an observed action and a possible attack path. Show the CIS framework version and report scope. Confirm that your workshop objects are removed.